

Automatically Distributing Quantum Programs

ANONYMOUS AUTHOR(S)

Current quantum computers cannot sustain the large entangled qubit groups required for practical quantum algorithms such as Shor’s factoring algorithm. Manually partitioning a sequential quantum program across distributed quantum processing units is error-prone and requires reasoning about quantum resource constraints, such as no-cloning and entanglement preservation, that have no classical analogue. We present **AutoDisQ**, the first system to automatically and correctly rewrite a sequential quantum program into an optimized distributed version, with soundness verified in Rocq. The language of **AutoDisQ** is a formal model of distributed quantum processors that combines the concepts of the Chemical Abstract Machine (CHAM) and Markov Decision Processes (MDP) with the objective of providing clear quantum distributed behaviors. **AutoDisQ** enables programmers to write quantum algorithms sequentially and deploy them across networked QPUs, bringing large-scale quantum computation closer to practical reach on near-term hardware.

1 Introduction

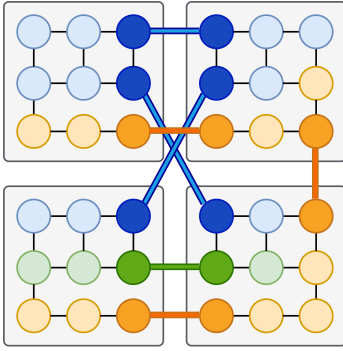


Fig. 1. A distributed QPU structure with four QPUs, executing three programs (in three different colors). The highlighted qubits are communication qubits, and the double line indicates communication between qubits from different QPUs.

Quantum computing has shown significant promise for achieving quantum advantage by enabling the design of algorithms that are substantially faster than their classical counterparts. However, near-term intermediate-scale quantum (NISQ) devices face serious challenges in scaling up to execute practical quantum applications [Caleffi et al. 2022; Tang and Martonosi 2024]. A key limitation is that the quantum entanglement—a critical resource that powers many quantum algorithms—is constrained by the small size of current machines. For example, while implementing Shor’s algorithm would require approximately 5,000 coherent, entangled (logical) qubits, existing single-location quantum computers can sustain only about 50 such qubits.

To overcome the scalability limitations of current quantum computers, the next generation of quantum computing architectures is moving towards Distributed Quantum Computing (DQC), which is built on the foundation of interconnected Quantum Processing Units (QPUs) [Chen et al. 2023; Chu et al. 2024; Inc et al. 2024; IonQ 2024; Main et al. 2024; Swayne 2024], as illustrated in Figure 1. A QPU typically consists of a local cluster of qubits (circle nodes in the figure) that can achieve high levels of entanglement, but the size of such entangle-

ment remains limited by the hardware architecture. To interconnect multiple QPUs, photonic qubits and links (shown as darker nodes and dashed lines)¹ enable entanglement sharing across different QPUs. These photonic qubits are often referred to as communication qubits, or “commq” for short. By linking QPUs in this way, quantum computers can create large-scale entangled states necessary to run complex quantum algorithms. Additionally, each QPU can perform software parallelism to improve performance; i.e., a sequential quantum circuit is split into parallel components applied to different qubits.

Based on the above QPU scheme, it is necessary to develop a programming paradigm for distributed quantum programs. This faces two challenges: 1) we want to automatically rewrite a

¹This approach uses photonic channels, such as Bell pairs, to distribute entanglement between physically separate QPUs.

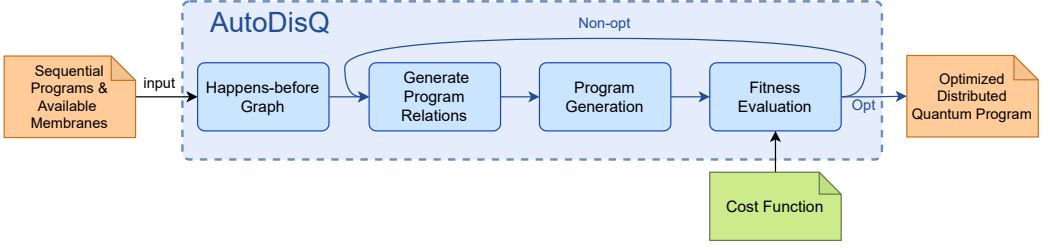


Fig. 2. AutoDisQ Algorithm Flow.

sequential quantum program to a distributed version with a correctness guarantee, 2) the automated rewrites need to be efficient and satisfy some fitness criteria, and 3) we need to be able to describe a distributed quantum program, so that the system can output meaningful distributed quantum programs.

We propose the first framework, AUTODisQ, for automatically rewriting a sequential quantum program into a distributed one. The core is a distributed quantum programming language framework, DisQ, that models DQC systems, and the AUTODisQ algorithm (Figure 2) uses the model to produce distributed quantum programs, enabling automated rewriting of a sequential quantum program based on a classical process algebraic framework. Our emphasis is on managing quantum resources, such as qubit entanglement, across different QPUs and supporting the development of distributed quantum programs, acknowledging qubit resource limitations.

Figure 2 shows the AUTODisQ algorithm flow. The algorithmic inputs are a sequential quantum program, a set of target membranes, and a fitness function that determines how well a generated distributed quantum program satisfies user criteria. The main algorithm is a loop that searches for the generated program with the optimized fitness.

In our framework, we also discuss the similarities between classical message passing and quantum remote communication, identify additional constraints of quantum communication, and model quantum communication, a.k.a., quantum teleportation, as a special kind of classical message-passing paradigm. To track qubit information transmission across different QPU locations, we also use a special kind of type construct, *loci*. This extension enables us to reuse classical simulation techniques to reason about correctness, ensuring that all quantum operations adhere to physical constraints. To model the system, we develop a simple language based on Chemical Abstract Machine (CHAM) [Berry and Boudol 1992] to represent different QPUs and Markov decision process (MDP) to express the DQC semantics.

AUTODisQ aims to enable the automated distribution of sequential quantum programs, enabling the execution of non-trivial quantum programs in the near term. We have the following contributions.

- We develop the AUTODisQ language (Sections 3.2 and 3.3) with its syntax and MDP-based semantics to capture DQC behaviors. The language originates from DisQ [Chang et al. 2025], but we simplify the design for message communication, channel creation, operation classification, and membranes. We intend to use a simplified language to capture the essence of quantum process algebra.
- By examining the DQC constraints, we develop an AUTODisQ (Section 4) that automatically and correctly rewrites a sequential quantum program into its distributed version. To the best of our knowledge, AUTODisQ is the first framework to perform automated rewrites of a quantum sequential program into a distributed one.

- We prove the correctness (Theorem 4.3) of the AUTODisQ algorithm in Rocq and show that for any sequential quantum program, AUTODisQ can correctly rewrite it to a distributed version. The correctness proof is done via a (bi)-simulation relation we constructed based on our language.
- We experiment AUTODisQ with 8 programs (Section 5), with different qubit settings, showing the utility of AUTODisQ in generating distributed quantum programs based on their sequential versions.
- We also discuss how to improve the effectiveness of AUTODisQ and link it to traditional software parallelism within a single QPU, with case studies (Section 6).

2 Background

Here, we provide background on quantum states and computation.

Quantum Data and State used in AUTODisQ. A quantum datum (often called a quantum state) consists of one or more qubits. A single qubit can be represented as a two-dimensional vector $\begin{pmatrix} z_1 \\ z_2 \end{pmatrix}$, where z_1 and z_2 are complex amplitudes with $|z_1|^2 + |z_2|^2 = 1$. Using Dirac notation, this is written as $z_1 |0\rangle + z_2 |1\rangle$, with $|0\rangle$ and $|1\rangle$ as the computational basis states. When both z_1 and z_2 are non-zero, the qubit is in a superposition of $|0\rangle$ and $|1\rangle$. Multi-qubit data is constructed via the tensor product, e.g., $|0\rangle \otimes |1\rangle = |01\rangle$. However, not all multi-qubit states can be separated into tensor products; some are entangled states, such as the Bell pair $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

Basic Terms:

Nat	$i, j, m, n \in \mathbb{N}$	Amplitude	$z \in \mathbb{C}$	Bitstring	$d \in b^*$
Bit	$b ::= 0 \mid 1$	Basis Vector	$\beta ::= (d\rangle)^*$	Location	$l, r, u \in L$
Variable	x, y	Classic Chan	a	Quantum Chan Name	c

Kinds and Classical/Quantum Data:

Kind	$g ::= C \mid Q(n)$
Classical Scalar Data	$v ::= d \mid \theta$
Basic Ket	$\eta ::= z\beta$
Quantum Data	$\theta ::= \sum_{j=0}^m \eta_j$

Quantum Loci, Environment, and States

Qubit Array Range	$q ::= x[n]$		
Locus	$\kappa ::= \overline{\langle q \rangle}_l$	concatenated op	$\sqcup$
Quantum State (Heap)	$\varphi ::= \kappa : \theta$	concatenated op	$\sqcup$

Fig. 3. Quantum data representation. Loci are finite lists, while states are finite sets. The operations after "concatenated op" refer to the concatenation operations for loci and quantum program states. Term a refers to classical channels, and c is a quantum channel name. $x[n]$ is the indexed n of qubit array x ; a range $x[n, m)$ is a syntactic sugar of $x[n] \sqcup x[n+1] \sqcup \dots \sqcup x[m-1]$.

Figure 3 demonstrates the quantum state representation in this paper. We anticipate two kinds of data: scalar (C) and quantum ($Q(n)$, representing n qubit arrays). For simplicity, variables and locations are in distinct categories. We permit all kinds of scalar data valuation, as we mainly refer to them as bitstrings (a) in this paper. A $Q(n)$ kind quantum data valuation is represented using Dirac notation $\sum_{j=0}^m z_j \beta_j$, where m is the number of basis-kets in the quantum data; each basis-ket contains an amplitude z_j and a basis vector β_j , with $|\beta_j| = n$ for all j , representing an n -qubit quantum array. Conceptually, quantum data are stored as a heap (a quantum state $\varphi \triangleq \kappa : \theta$), partitioned into regions described as *loci* (κ); each region contains possibly entangled qubits, with the guarantee that cross-locus qubits are not entangled. Figure 4 describes several scenarios for connecting ranges in a locus. We use loci to group possibly entangled qubits and ensure qubit

disjointness; each locus can be viewed as a chain of disjoint region segments labeled with explicit information about the location of local state variables, e.g., $\langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r$ suggests that the two qubits, both named $c[0]$, in locations l and r are possibly entangled. Note that the l notation in loci captures location information. Each locus contains a list of *disjoint ranges* (s), each represented by $x[n, m)$ —an in-place array slice selected from n to m (exclusive) in a physical qubit array x (always being Q kind). Ranges in a local locus are pairwise disjoint, written as $s_1 \sqcup s_2$.

Each element in a quantum state φ maps a locus κ to a quantum datum $\sum_{j=0}^m z_j |d_j\rangle$, with $|\kappa| \leq |d_j|$, i.e., the qubit length in κ might be less than the bit-string length in a basis-vector $|d_j\rangle$. Essentially, a locus κ acts as a sequence of pointers pointing to entangled qubits, with information partly stored in $|d_j\rangle$. We call the corresponding basis vector bits of qubits or locus fragments for a datum (or a basis-ket set) as the *qubit's/locus's position bases* of the datum (or the basis-ket set).

Quantum Computation. Quantum computation applies unitary gates (e.g., Hadamard, controlled-not) to evolve the state of qubits. A gate is represented by a unitary matrix U that acts on the qubit state vector $|\psi\rangle$, producing a new state $U|\psi\rangle$. For example, the Hadamard gate $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, in Figure 5, transforms computational basis states into superpositions: applying H to $|0\rangle$ yields $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, and applying H to $|1\rangle$ yields $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. These operations are composed in quantum circuits to implement computations, where each wire denotes a qubit, and each gate denotes a transformation applied at a specific time.

There are multi-qubit operations, applying to multiple qubits, e.g., $w \sqcup v \leftarrow \text{CU}(\lambda t. |v^{2^i} * t \% N\rangle)$ controls on w and applies the quantum arithmetic operation $(\lambda t. |v^{2^i} * t \% N\rangle)$ to the qubit array v . This paper uses this prefixed notation, which is a syntactic sugar of a traditional quantum control gate notation $w \sqcup v \leftarrow \text{CU}(w) \{(\lambda t. |v^{2^i} * t \% N\rangle)(v)\}$, controlling on w and applying $(\lambda t. |v^{2^i} * t \% N\rangle)$ to v . In quantum arithmetic operations, a quantum qubit array is viewed as a linear sum of many different possibilities, each of which is represented as a bitstring (basis-ket), e.g., $\lambda t. |f(t)\rangle$ takes in a qubit array t and applies the arithmetic function $f(t)$ to every bitstring state of t .

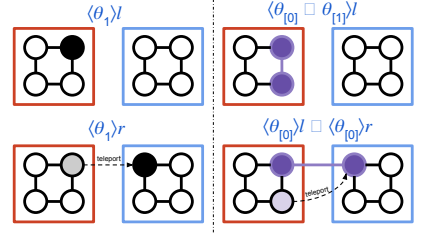


Fig. 4. Examples of how locus works

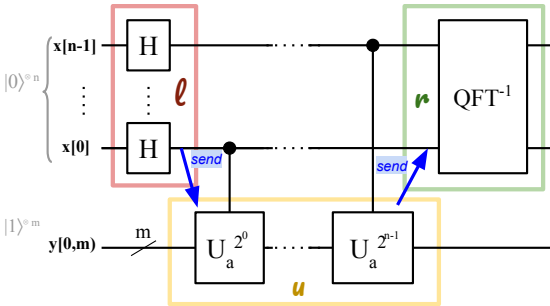


Fig. 5. The order finding of Shor's algorithm, possibly splitting into two pieces l , u , and r , executable in three QPUs, connected via photonic links.

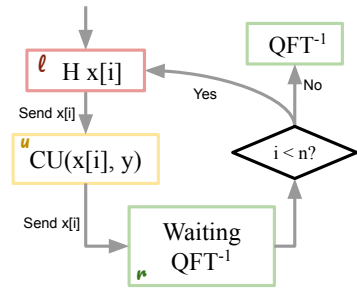


Fig. 6. Distributed Transitions. A quantum channel $c(1)$ (state: $c[0]$) connects l and u , and $c'(1)$ (state: $c'[0]$) connects u and r .

Measurement collapses the quantum state to a classical outcome with a probability determined by the amplitudes. For instance, measuring $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ yields $|0\rangle$ or $|1\rangle$, each with probability $\frac{1}{2}$. After measurement, the state irreversibly collapses to the observed basis state, and quantum coherence is lost.

3 Overview via Distributed Shor’s Algorithm

This section motivates the challenges of distributing quantum programs and previews how AUTODisQ addresses them, using Shor’s algorithm as a running example. Shor’s algorithm finds a nontrivial factor of a large number N . Its core quantum component (Figure 5) uses two registers, x (storing the candidate order) and y (storing modular multiplication results). To factorize a 300-bit number, the order finding step requires at least 900 entangled qubits (roughly 300 for x and 600 for y), yet today’s single-location quantum computers can sustain only about 50 coherent, entangled qubits. Distributed quantum computing offers an escape: split the program across multiple QPUs connected by photonic links.

Suppose we naively place the x register on QPU₁ and y on QPU₂. The controlled-unitary $CU(x[i], y[0, n])$ requires *both* registers, but they now reside on different machines. We must transmit, by quantum teleportation, $x[i]$ to QPU₂, but quantum teleportation *destroys* the sender’s copy (no-cloning), *consumes* a pre-shared Bell pair, and must *preserve* any entanglement $x[i]$ has with other qubits. Getting any of these wrong silently corrupts the computation. Concretely, these challenges arise:

- (1) **Deciding what goes where.** Operations have data dependencies (a happens-before relation). Assigning them to QPUs must respect these dependencies while balancing qubit usage across membranes.
- (2) **Inserting communication correctly.** When consecutive operations on the same qubit land on different QPUs, we must insert quantum teleportation pairs. Unlike classical message passing, quantum communication is destructive and must preserve entanglement with third-party qubits.
- (3) **Optimizing the partition.** Teleportation is expensive: roughly 60× slower than a local gate on current hardware. A brute-force search over partitions is factorial in the number of operations; we need a tractable strategy that still finds good solutions.

In classical distributed computing, a process can copy a datum and send it to a remote node. In quantum computing, the no-cloning theorem forbids copying: sending a qubit is a *destructive transfer* that requires a pre-shared entangled channel (Bell pair) and physically relocates the quantum information. Any entanglement the qubit had with other qubits must be preserved through the transfer. This makes the problem fundamentally harder than classical program partitioning and is the reason DisQ models quantum channels as one-time resources consumed upon use.

Not every partition is useful: if we assign both the Hadamard and CU operations to membrane l , then l must hold all n qubits of x and all n qubits of y simultaneously, defeating the purpose of distribution. Conversely, if we split individual gate pairs across membranes (as can happen with a naïve algorithm), we insert far more communication pairs than necessary, overwhelming the benefit. AUTODisQ searches for a partition that balances qubit usage per membrane against communication cost.

We now preview the AUTODisQ approach on Shor’s order-finding circuit (Figures 5 and 6). The program structure is based on a quantum process algebra work DisQ [Chang et al. 2025], where

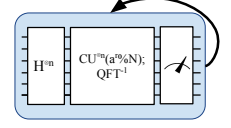


Fig. 7. Shor’s Flow

Unitary Expr	μ	
Bool Expr	B	
Action	U	$::= \partial \bar{q} \mid \bar{q} \leftarrow \mu \mid a!d \mid c!q \mid a?(x) \mid c?(q)$
Process	R, T, M, N	$::= 0 \mid U.R \mid \text{if } (B) R \text{ else } T \mid x \leftarrow \mathcal{M}(\bar{s}).R$
Membrane	P, Q	$::= \{\!\{R\}\!\}_l$

Fig. 8. Program syntax; syntactic sugar: $\text{if } (B) \{R\}.T = \text{if } (B) R.T \text{ else } T$.

AUTODisQ is a generalized framework suitable for any multi-threaded quantum programming languages.

3.1 Distributed Shor's Algorithm in DisQ

We can apply AUTODisQ to split the order-finding circuit into three membranes (l , u , and r) in Figure 5, each executed on a separate QPU, and use quantum networking to communicate between them. Here, AUTODisQ aims to place the operations for the x and y registers on different QPUs. Furthermore, each qubit in the x registers is applied by an individual Hadamard operation without any connection, indicating that these qubits can perform gate operations in sequence to ensure that we only manipulate one qubit in the l location at a time. Below is an example of a distributed single-step order-finding operation across three membranes, as shown in Figure 6.

Example 3.1 (One Step Distributed Shor's Algorithm). We show the three membranes, l , u , and r , for performing one step computation in Figure 5, assuming that we have a 1-qubit quantum channel c between l and u , as well as c' between u and r .

$$\{\!\{x[i] \leftarrow H.c!x[i].0\}\!\}_l, \{\!\{c?(w).w \boxplus y[0, n] \leftarrow CU(\lambda t. |v^{2^i} * t \% N|).c'!w.0\}\!\}_u, \{\!\{c'?(q) \dots\}\!\}_r$$

Assume that x and y registers have n qubits and $x[i]$ refers to the i -th qubit in x and $y[0, n]$ to refer to the range $[0, n]$ of the locus $y[0, n]$. We restructure the operations in membranes l and u (Figures 5 and 6), to be a repetition of two operations $H(x[i])$ and $CU(\lambda t. |v^{2^i} * t \% N|)(x[i], y[0, n])$ for $i \in [0, n]$ (DisQ uses the program syntax format $x[i] \leftarrow H$ and $x[i] \boxplus y[0, n] \leftarrow CU(\lambda t. |v^{2^i} * t \% N|)$). Each single step applies a Hadamard gate to $x[i]$ in l and transmits $x[i]$'s quantum information to u , via the channel c , so that a control gate ($x[i] \boxplus y[0, n] \leftarrow CU(\lambda t. |v^{2^i} * t \% N|)$) can be applied to. The operation controls on $x[i]$ and applies a modulo-multiplication to $y[0, n]$. We then utilize the c' channel to transmit $x[i]$ again to the membrane r .

3.2 Syntax for the DisQ Language and Modeling Distributed Quantum Programs

In AUTODisQ, a distributed quantum program is described using a multiset of location-specific quantum processes, whose syntax (Figure 17) is captured by a membrane ($\{\!\{R\}\!\}_l$). A process R , localized to a membrane, can be understood as a sequence of communication and quantum actions (U). We permit process algebraic message transmission operations, e.g., $\alpha!v$ and $\alpha?(y)$; they are the only communication actions that can perform direct message passing between different membranes, which can perform both classical and quantum message passing, depending on if the channels are classical or quantum; quantum and classical communication similarity is described shortly below.

A quantum operation can be a new blank ($|0\rangle$) qubit array generator $\partial x(n)$, and a unitary operation $\kappa \leftarrow \mu$, applying a unitary operation μ to a local locus κ . In DisQ, we abstract away the detailed implementation of μ and assume that they can be analyzed by some systems describing quantum unitary circuits, e.g., VOQC [Hietala et al. 2023]. As shown in Section 2, we require a control operation to show the control node via the $\boxplus$ separator in a locus, as $w \boxplus v \leftarrow CU(\lambda t. |v^{2^i} * t \% N|)$. We also permit a classical conditional $\text{if } (B) R \text{ else } T$, where the expression B is an arbitrary classical Boolean expression, implemented using bit-arithmetic, i.e., 1 as true and 0 as false; as well as measurement operations, $x \leftarrow \mathcal{M}(\kappa).R$, measuring κ and storing the vector output to x .

$$\begin{array}{c}
\text{S-NEW} \\
\frac{x[0, n] \cap \text{dom}(\varphi) = \emptyset}{(\varphi, \{\partial x(n).R\}_l) \xrightarrow{L,1} (\varphi \uplus \{\bigoplus_{j=0}^{n-1} \{\langle x[j] \rangle_l : |0\rangle\}, \{R\}_l)} \\
\\
\text{S-IFT} \\
(\varphi, \{\text{if } (1) R \text{ else } T\}_l) \xrightarrow{L,1} (\varphi, \{R\}_l) \\
\\
\text{S-OP} \\
(\varphi \uplus \{\langle \bar{q} \rangle_l \sqcup \kappa' : \theta\}, \{\bar{q} \leftarrow \mu.R\}_l) \xrightarrow{L,1} (\varphi \uplus \{\langle \bar{q} \rangle_l \sqcup \kappa' : \llbracket \mu \rrbracket^{\bar{q}} \theta\}, \{R\}_l) \\
\\
\text{S-IF} \\
(\varphi, \{\text{if } (0) R \text{ else } T\}_l) \xrightarrow{L,1} (\varphi, \{T\}_l) \\
\\
\text{S-MEA} \\
\frac{p = \sum_j |z_j|^2}{(\varphi \uplus \{\langle \bar{q} \rangle_l \sqcup \kappa' : \sum_j z_j |d\rangle |d_j\rangle + \theta \langle \bar{q}, d \neq \kappa \rangle\}, \{x \leftarrow \mathcal{M}(\bar{q}).R\}_l) \xrightarrow{L,p} (\varphi \uplus \{\kappa' : \sum_j \frac{z_j}{\sqrt{p}} |d_j\rangle\}, \{R[d/x]\}_l)} \\
\\
\text{S-COMP} \\
\frac{(\varphi, P) \xrightarrow{\xi, p} (\varphi', P')}{(\varphi, P, Q) \xrightarrow{\xi, p} (\varphi', P', Q)} \quad \text{S-COMMA} \\
(\varphi, \{a!v.R\}_l, \{a?(x).T\}_r) \xrightarrow{L,r,1} (\varphi, \{R\}_l, \{T[v/x]\}_r) \quad \text{S-END} \\
(\varphi, \{\bar{0}\}_l) \xrightarrow{L,1} (\varphi, \emptyset) \\
\\
\text{S-COMM} \\
\frac{\langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r \cap \text{dom}(\varphi \cup \{\langle x[n] \rangle_l\}) = \emptyset \quad \langle x[n] \rangle_r \notin \text{dom}(\kappa)}{(\varphi \uplus \{\langle x[n] \rangle_l \sqcup \kappa : \theta\}, \{c!x[n].R\}_l, \{c?(y[u]).T\}_r) \xrightarrow{L,r,1} (\varphi \uplus \{\langle x[n] \rangle_r \sqcup \kappa : \theta\}, \{R\}_l, \{T[x[n]/y[u]\}_r)}
\end{array}$$

Fig. 9. Semantic rules.

The quantum circuit component of a sequential Shor's algorithm, performing order finding, is in Figure 5, containing two registers x and y , where x represents the solution for the found order and y stores the modular multiplication constraint. The order finding part in a practical usage of Shor's algorithm requires x and y to have at least 900 qubits to factorize a 300 bit number, stored in x , indicating x needs 300 qubits for storage, and the y registers need 600 qubits to perform the modular-multiplication operations. NISQ is limited in terms of qubit count, and a possible solution is distributed quantum computation, which uses quantum networking techniques to connect multiple single-location QPUs; such communication can be described by quantum teleportation [Bennett et al. 1993; Rigolin 2005]. We can apply AUTODisQ to split the circuit into three membranes (l , u , and r) in Figure 5, each executed on a separate QPU, and use quantum networking to communicate between them.

Here, AUTODisQ aims to place the operations for the x and y registers on different QPUs. Furthermore, each qubit in the x registers is applied by an individual Hadamard operation without any connection, indicating that these qubits can perform gate operations in sequence to ensure that we only manipulate one qubit in the l location at a time. Below is an example of a distributed single-step order-finding operation across three membranes, as shown in Figure 6.

3.3 Semantics

We model the program semantics based on a Markov decision process in Figure 19, expressed as a labeled transition system $(\varphi, \bar{P}) \xrightarrow{\xi, p} (\varphi', \bar{Q})$, where ξ (either l or $l.r$) captures the membrane locations (l or $l.r$) participating in the nondeterministic choice of the transition, p represents the probability of the transition, and φ and φ' are the global pre- and post- quantum states described in Figure 3. The label in each rule indicates that the transition of membranes is an MDP, where we nondeterministically select membrane locations ξ to perform a single process semantics with the probability p .

Our program process is a sequence of actions, and the rules in Figure 19 define the semantics for a local action prefixed in a process. Rule S-NEW creates new n blank qubits for locus $\langle x[0, n] \rangle_l$. Rule S-OP applies a quantum unitary operation to a locus $\langle \bar{s} \rangle_l$'s quantum data. Here, the locus fragment

$\langle \bar{s} \rangle_l$ to which the operation is applied must be prefixed in the locus $\langle \bar{s} \rangle_l \sqcup \kappa'$ that refers to the entire quantum data θ . If not, we will first apply equivalence rewrites, explained in Section B.3 and [Li et al. 2024], to move $\langle \bar{s} \rangle_l$ to the front. With $\langle \bar{s} \rangle_l$ preceding the rest fragment κ' , the operation's semantic function $\llbracket \mu \rrbracket^n$ is then applied to $\langle \bar{s} \rangle_l$'s position bases in the quantum value θ . Rule S-IFT and S-IFF describe the semantics of classical conditionals.

We assume that the evaluation of the membrane set is *compositional* as S-COMP, i.e., every subset of the set can make a move. In the end, if every process in a membrane turns to $\emptyset$, rule S-END permits its termination. Rule S-COMMA performs a classical message communication inherited from traditional π -calculus [Milner et al. 1992], while S-COMMC performs a quantum message communication, assuming that the channel $c[0, n]$ is properly initialized as an n bitwidth Bell pair, as Φ_1 . Rules S-COMMA and S-COMMC transitions have labels $l.r.1$, meaning that the nondeterministic event happens across the l and r membranes. The probability 1 in the above three rules ensures that the transitions always happen.

Below, we show the transitions of executing the one-step Shor's algorithm in Theorem 3.1, for the membranes l and u ; the initial state is $|0\rangle$ for qubit $x[i]$ and $|1\rangle$ for the n -qubit array $y[0, n]$.

$$\begin{aligned}
 (1) \quad & \xrightarrow{\quad} \llbracket x[i] \leftarrow H.c!x[i].0 \rrbracket_l, \llbracket c?(w).w \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^i} * t \% N\rangle).c'!w.0 \rrbracket_u \quad \langle x[i] \rangle_l \sqcup \langle y[0, n] \rangle_u : \varphi_1 \\
 (2) \quad & \xrightarrow{l,1} \llbracket c!x[i].0 \rrbracket_l, \llbracket c?(w).w \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^i} * t \% N\rangle).c'!w.0 \rrbracket_u \quad \langle x[i] \rangle_l \sqcup \langle y[0, n] \rangle_u : \varphi_2 \\
 (3) \quad & \xrightarrow{l,u,1} \llbracket 0 \rrbracket_l, \llbracket x[i] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^i} * t \% N\rangle).c'!x[i].0 \rrbracket_u \quad \langle x[i] \sqcup y[0, n] \rangle_u : \varphi_2 \\
 (4) \quad & \xrightarrow{u,1} \llbracket 0 \rrbracket_l, \llbracket c'!x[i].0 \rrbracket_u \quad \langle x[i] \sqcup y[0, n] \rangle_u : \varphi_3 \\
 & \varphi_1 = |0\rangle|1\rangle \quad \varphi_2 = \sum_{b=0}^1 \frac{1}{\sqrt{2}} |b\rangle|1\rangle \quad \varphi_3 = \sum_{b=0}^1 \frac{1}{\sqrt{2}} |b\rangle|v^{2^i} * b \% N\rangle
 \end{aligned}$$

The effect of message communication occurs in the quantum state, where it transforms a quantum message between two membranes, mimicking quantum teleportation (Section C). In this paper, we call a pair of sending and receiving operations a *communication pair*. DisQ utilizes a locus structure $\langle x[i] \sqcup y[0, n] \rangle_l$ to indicate that two qubit arrays $x[i]$ and $y[0, n]$ are entangled and located in the membrane l , indicated by the operation $\sqcup$. After the communication transition in line (2), $x[i]$ is transformed to the membrane u . The physical meaning of the transformation is that we destroy the locus $\langle x[i] \rangle_l$ in the membrane l and reproduce it in the membrane u as $\langle x[i] \rangle_u$, while preserving all its information, using the same name $x[i]$ to indicate the phenomenon. Note that quantum entanglement is also a piece of information to be preserved. Assume that $\langle x[i] \rangle_l$ is entangled with $\langle x[0] \rangle_l$ before the message passing. After the communication, $x[i]$ is still entangled with $\langle x[0] \rangle_l$ and we can express the locus structure as $\langle x[0] \rangle_l \sqcup \langle x[i] \sqcup y[0, n] \rangle_u$, where $x[0]$ and $x[i]$ locate in different membranes.

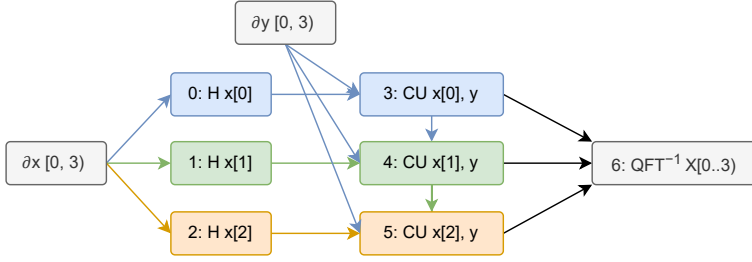
4 The AutoDisQ Algorithms

This section discusses the AutoDisQ algorithm in depth, with its flow in Figure 2. The algorithm's input includes a sequential quantum process R , a set of available membranes L , and a fitness function (fit). We first apply AutoDisQ to Shor's algorithm example and then detail AutoDisQ.

4.1 Automated Rewrite of the Sequential Shor's Algorithm

This section demonstrates AutoDisQ for automatically rewriting sequential quantum programs into distributed versions, using the sequential Shor's algorithm program (Figure 5) as an example. We first show the sequential version. A convenient notation for a recursive process (Rec) used in the paper is defined as $\text{Rec}(j, n, f) \triangleq \text{if } (j = n) \emptyset \text{ else } f(j). \text{Rec}(j+1)$.

Example 4.1 (Sequential Shor's Algorithm). We list the sequential Shor's algorithm for n -length x and y qubit arrays.

Fig. 10. *hfb* relation graph.**Processes:**

$$He(j) \triangleq Rec(0, n, x[j] \leftarrow H) \quad Ue(j) \triangleq Rec(0, n, x[j] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^j} * t \% N))$$

Membranes:

$$\partial x[0, n) . \partial y[0, n) . y[0, n] \leftarrow \lambda t. |t + 1\rangle . He(n) . Ue(n) . x[0, n] \leftarrow QFT^{-1} . 0$$

As shown in Figure 2, the AutoDisQ algorithm requires an input sequential program and a set of available target membranes for the rewrite. We also require users to define fitness to ensure the distributed version satisfies the resource limitations. The basic philosophy is that a sequential quantum program and its distributed version perform the same task, so they have the same set of operations.² In rewriting a sequential quantum program into a distributed one, we assign singleton operations to different membranes while preserving the program's correctness.

We use the quantum analogy of a *happens-before* (*hfb*) relation to represent sequential quantum program relations, stating whether or not an event must happen before another event. To understand the concept, we first introduce an operation list (*os*) for the input sequential program that contains all program operations. The operation list for Theorem 4.1 is given as follows.

$$os \triangleq [\partial(x[0, n)), \partial(y[0, n)), y[0, n] \leftarrow \lambda t. |t + 1\rangle, x[0] \leftarrow H, \dots, x[n-1] \leftarrow H, \\ x[0] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^0} * t \% N)), \dots, x[n-1] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^{n-1}} * t \% N)), x[0, n] \leftarrow QFT^{-1}]$$

The operations in the list are ordered based on the execution in the sequential program, and we construct the *hfb* relation based on the operation list indices (ranks). An example *hfb* relation generated based on Theorem 4.1 for $n = 3$ is drawn as a directed graph in Figure 10. The indices 0 and 1 refer to the qubit initialization for $x[0, n)$ and $y[0, n)$; 2 to 4 refer to the Hadamard operations $x[j] \leftarrow H$ for $j \in [0, 3)$, in the *He* process; the indices 5 to 7 refer to the CU operations $x[j] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^j} * t \% N))$ for $j \in [0, 3)$, in the *Ue* process; and the index 8 refers to the inverse QFT operation ($x[0, 3] \leftarrow QFT^{-1}$). In the *hfb* relation, all Hadamard operations ($x[j] \leftarrow H$) can be run in parallel, while they must happen before the operations $x[j] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^j} * t \% N))$, which must happen before the inverse QFT operation.

Once the *hfb* relation is constructed, the AutoDisQ algorithm executes a loop to search for a distributed quantum program satisfying three constraints: 1) the constraint of the *hfb* relations, 2) the constraint of available target membranes representing the set of all possible distributed machines, each labeled with a membrane name, and 3) the constraint of the fitness function determining the resource limitation that users impose on the system. Each loop consists of three key steps: randomly generating distributed program relations, constructing the actual distributed program, and evaluating it using a predefined fitness function.

²Once a distributed program is formed, there might be additional circuit optimization chances, beyond the paper's scope.

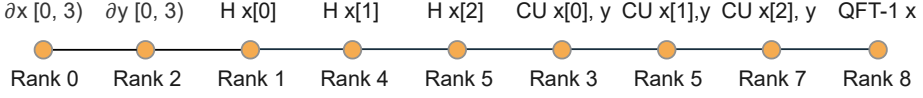
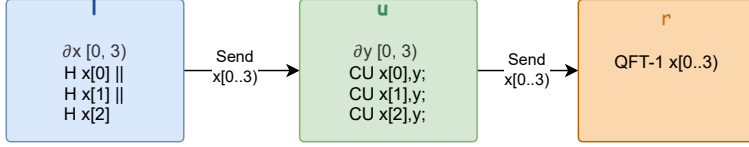
Fig. 11. An example candidate *seq* relation.

Fig. 12. An example operation membrane assignment.

We now demonstrate the three key steps in a loop. We first generate distributed program relations, which take the above *fib* relation and operation list as input and produce a sequenced-before (*seq*) relation and a membrane assignment for operations. The generation of the *seq* relation fixes the execution order of an operation in the operation list. To represent *seq*, we assign a rank number j to each operation in the operation list os , representing that the operation is executed as the j -th one in the whole distributed program execution. For example, we generate the *seq* relation by assigning the rank numbers for the above operation list in Figure 11, as follows.

$$seq \triangleq [\partial(x[0, n]) : 0, \partial(y[0, n]) : 2, x[0] \leftarrow H : 1, x[1] \leftarrow H : 4, x[2] \leftarrow H : 5, x[0] \sqcup y[0, 3) \leftarrow CU(\lambda u. |v^{2^0} * u \% N) : 3, x[1] \sqcup y[0, 3) \leftarrow CU(\lambda u. |v^{2^1} * u \% N) : 6, x[2] \sqcup y[0, 3) \leftarrow CU(\lambda u. |v^{2^2} * u \% N) : 7, x[0, 3) \leftarrow QFT^{-1} : 8]$$

In the above *seq* relation, the operations $x[0] \leftarrow H$ and $x[1] \sqcup y[0, 3) \leftarrow CU(\lambda t. |v^{2^1} * t \% N)$ are executed as the first (1) and second (6) operations in the system, respectively. Note that the above *seq* relation is only one of the possible ways, randomly generated by AutoDisQ.

AutoDisQ also automatically generates a membrane assignment (*mo*), assigning membrane names to operations, i.e., we need to define the specific membrane for each operation execution. For example, one possible *mo* for the above example is listed as follows, visualized in Figure 12.

$$mo \triangleq [\partial(x[0, n]) : l, \partial(y[0, n]) : u, x[0] \leftarrow H : l, x[1] \leftarrow H : l, x[2] \leftarrow H : l, x[0] \sqcup y[0, 3) \leftarrow CU(\lambda t. |v^{2^0} * t \% N) : u, x[1] \sqcup y[0, 3) \leftarrow CU(\lambda t. |v^{2^1} * t \% N) : u, x[2] \sqcup y[0, 3) \leftarrow CU(\lambda t. |v^{2^2} * t \% N) : u, x[0, 3) \leftarrow QFT^{-1} : r]$$

The assignments (l and u) to the qubit initialization operations ($\partial(x[0, n])$ and $\partial(y[0, n])$) represent the initialization of the two qubit ranges in the two different membranes, respectively. We apply the Hadamard operations ($x[j] \leftarrow H$ for $j \in [0, 3)$) in the membrane l , the CU operations ($x[j] \sqcup y[0, 3) \leftarrow CU(\lambda t. |v^{2^j} * t \% N)$ for $j \in [0, 3)$) in the membrane u and the inverse QFT operation in the membrane r .

The simplest algorithm can be a brute force one to generate the above *seq* relation and membrane assignments randomly, evaluate different cases, and select the one with the best fitness score. However, such an algorithm might not be effective. We apply optimization strategies in our algorithm based on our *fib* relation; see Section 4.2.

After generating *seq* and *mo*, we generate a DisQ program based on the information. Conceptually, we first generate a sequence of operations based on the rank order, assigned to each operation in the generated *seq* relation. We then partition the sequence into subsequences based on the membrane assignment and place them in the corresponding membrane. As an example of Shor's algorithm,

we first consider the operation $\partial x[0, n]$, which has rank 0. We place it in membrane l according to its membrane assignment. We then deal with the operation $x[0] \leftarrow H$, having rank 1.

The above procedure generates a DisQ program without having cross-membrane communications, i.e., we need to relocate qubits across different membranes. As an instance, based on the above program relations, the operation $x[0] \leftarrow H$ happens at the membrane l , and the same qubit $x[0]$ needs to transmit to the membrane u , to be applied by the operation $CU(\lambda t. |v^{2^0} * t \% N\rangle)$, and then transmit to the membrane r , to be applied by the inverse QFT operation. This indicates that we need to relocate $x[0]$ from the membrane l to u after the Hadamard operation and relocate it again from u to r after the CU operation. To enable each qubit relocation, we need to insert a quantum communication pair. Eventually, we generate a DisQ program based on the program relation map as follows, when $n = 3$. Here, we use the recursive operator $Rec(j, n, f) \triangleq \text{if } (j = n) 0 \text{ else } f(j) . Rec(j+1)$ in the beginning of Section 4.1.

Example 4.2 (Distributed Shor's Algorithm Program). The program has three membranes l , u , and r . Initially, l and u hold n -qubit x and y qubit arrays (loci), respectively, while the membrane r does not hold any qubits. The membranes l and u share n quantum channels c_j , while the membranes u and r share n quantum channels c'_j , for $j \in [0, n]$.

Processes:

$He(j) \triangleq x[j] \leftarrow H . c_j ! x[j] . 0$
 $Me(j) \triangleq c_j ? (x'[j]) . x'[j] \sqcup y[0, n] \leftarrow CU(\lambda t. |v^{2^j} * t \% N\rangle) . c'_j ! x'[j] . 0$
 $Ed(j) \triangleq c'_j ? (x''[j]) . 0$

Membranes:

$\{\{\partial x[0, n] . Rec(0, n, He)\}_l, \{\{\partial y[0, n] . y[0, n] \leftarrow \lambda t. |t + 1\rangle . Rec(0, n, Me)\}_u, \{\{Rec(0, n, Ed) . x''[0, n] \leftarrow QFT^{-1} . 0\}\}_r$

The He process performs a Hadamard operation and sends it out from the membrane l to u , the Me process receives a qubit from l , applies a CU operation and sends the qubit out to the membrane r , and the Ed process receives a qubit from u . The membranes l , u , and r repeat the He , Me , and Ed processes n times, respectively. Eventually, we apply an inverse QFT operation in the membrane r .

After a DisQ program is automatically generated, we need to evaluate the program for a specific fitness, which can be defined by users. In rewriting a sequential quantum algorithm, we mainly focus on two criteria: 1) minimizing the maximum number of qubits used in a membrane, and 2) minimizing the number of quantum communication pairs being inserted. As in Section 1, it is ideal to rewrite quantum operations executed in different QPUs, so that each QPU has a balanced qubit usage. On the other hand, quantum message communication via quantum teleportation is actually costly. Having too many communication pairs might overturn the benefit of distributed quantum computation. The ultimate fitness is a trade-off between the two criteria. In the above 6 qubit Shor's algorithm example, users can define the fitness to have a threshold of having maximum qubit usage less than 4 qubits and maximally 6 quantum communication pairs being generated. After running AutoDisQ, we generated the program in Theorem 6.3, which satisfies the above user-defined fitness.

4.2 The Distributing Algorithm

We discuss the algorithm, illustrated in Algorithm 1, for rewriting a sequential quantum process (R) into a DisQ distributed quantum program Q . The happens-before $\hbar b$ relation can be trivially generated (Algorithm 1 line 3) via traditional $\hbar b$ relation techniques [Lampert 1978; Tanenbaum and van Steen 2017].

Algorithm 1 AutoDisQ Algorithm

```

1: Input:  $R, L$  //a sequential process and membrane location set.
2: Output:  $Q$ 
3:  $hb \leftarrow \text{gen\_hp}(R)$ 
4:  $os \leftarrow \text{gen\_os}(R)$ 
5:  $S \leftarrow \text{gen\_mem}(\text{gen\_seq}(hb, os), L)$  //creating candidate set by generating seq and membrane assignments.
6:  $z_{\min} \leftarrow \top$ 
7:  $Q \leftarrow \perp$ 
8: do
9:    $(seq, mo) \in S$ 
10:   $P \leftarrow \text{gen\_prog}(seq, mo)$ 
11:   $S \leftarrow S \setminus \{(seq, mo)\}$  //accumulating examined program relation.
12:   $z \leftarrow \text{fit}(P)$ 
13:  if  $z < z_{\min}$  then //if the result is the new minimal.
14:     $Q \leftarrow P$ 
15:     $z_{\min} \leftarrow z$ 
16:  end if
17: while  $S \neq \emptyset$  //until no more possible seq cases to examine.
18: Return  $Q$ 

```

Program Relation Generation. With the hb relation and an operation list (os), we randomly produce a sequenced-before relation seq (Algorithm 1 line 9) and membrane assignment for operations (Algorithm 1 line 10), where seq determines the execution order of operations, the operation membrane assignment (mo) maps operations to membrane labels, selecting the membrane location for the operations. Given an operation list, one can generate a seq graph by assigning the rank numbers for each operation in the operation list.

In generating a seq relation with n operations, there are $n!$ possible ways of constructing seq . For each possible seq , we also need a membrane assignment mo , assigning operations to different membranes, to turn it into a program, as shown in Algorithm 1 line 5. Given N qubits and m different membranes, there are a total of $m * N * n!$ different combinations to construct a distributed quantum program. Therefore, the brute-force algorithm is ineffective because there may be too many cases to consider when the sequential quantum program is extensive. We implement two optimizations to significantly reduce the number of cases.

First, for the seq generation (gen_seq), note that quantum operations can typically be partitioned into several groups, each containing similar operations. For example, Shor's algorithm in Theorem 4.1 can be viewed as four different groups: two qubit creation operations, n H operations, n controlled modular multiplication operations, and a QFT operation. Each group of operations is similar, differing only in which qubits they are applied to. The simple brute-force algorithm for generating seq is to perform permutations of operation ranks in os with respect to hb . To reduce the number of permutations, AutoDisQ fixes the operation order in each group above, while we shuffle the operation orders across different groups. For example, the generated seq for Theorem 4.1 ensures that Hadamard operations ($x[j] \leftarrow H$) happen before CU operations ($x[j] \boxplus y[0, n] \leftarrow \text{CU}(\lambda t. |v^{2^j} * t \% N|)$), and we ensure the Hadamard operations are ordered based on the j incrementally from 0 to n . We only allow Hadamard and CU operations to be shuffled.

The second optimization is related to the membrane assignments. The qubit creation operations essentially assign qubits to a particular membrane, e.g., $\partial x[0, n]$ and $\partial y[0, n]$ in Theorem 4.1 are assigned to the membranes l and u , respectively. Apparently, we can apply Hadamard operations

Algorithm 2 Program Generation Algorithm

```

1: Input:  $seq, mo$ 
2: Output:  $P$ 
3:  $P \leftarrow \perp$ 
4:  $cur \leftarrow qloc(mo)$  //the current membrane locations of qubits.
5:  $name \leftarrow \emptyset$ 
6: while  $seq \neq \emptyset$  do
7:    $op, seq \leftarrow next(seq)$  //partition the next op from seq.
8:    $l \leftarrow mo(op)$ 
9:    $S \leftarrow diff\_mem(cur(locus(op)), l)$  //S contains operations in different membranes than l.
10:  if  $S \neq \emptyset$  then
11:     $P \leftarrow insert\_send(P, locus(S), name)$ 
12:     $P, name \leftarrow insert\_rev(P, locus(S), l, name)$ 
13:     $cur \leftarrow cur\_up(cur, locus(S))$  //update the current qubit assignment.
14:  end if
15:   $P \leftarrow place(P, op, l, name)$ 
16: end while
17: Return  $P$ 

```

($x[j] \leftarrow H$) to the membrane l since the $x[j]$ qubit is located in l . For the CU operations, we probably place it in the membrane u , since most of the qubits in the locus $x[j] \sqcup y[0, n)$ belong to the membrane u .

Our membrane assignment algorithm (`gen_mem`) performs a similar optimization: we keep a map storing the current qubit locations, e.g., in the beginning, we think $x[0, n)$ and $y[0, n)$ are located in l and u , respectively. We then look for the operation (μ) one by one based on the *seq* order. If the qubits, mentioned in μ , are all located in a membrane l , we assign μ to l , e.g., the Hadamard operations above.

If the qubits are partially located in a membrane l beyond a threshold, e.g., more than 60%, we assign μ to l and also move all μ 's qubits to l , indicating that we also need to insert communication pairs to transmit the qubits from the current qubit locations to l . For example, the CU operation has most of its qubits ($x[j] \sqcup y[0, n)$) in membrane u ($y[0, n)$ is in u), we then place it to u , and insert a communication pair to transmit $x[j]$ from l to u . Finally, if the qubits are partially located in a membrane l but less than a threshold or if l contains qubits more than another threshold, we assign μ to l and also randomly assign it to another membrane r with the least current qubit usage.

By applying these optimizations, we can significantly reduce the number of programs to consider. By rewriting Shor's sequential algorithm as a distributed one, we need to consider only a significantly smaller set of cases than in the original algorithm.

Generated Distributed Candidates. With the generated *seq* graph and membrane assignments *mo*, we generate a DisQ distributed candidate P , following Algorithm 2, representing the function `gen_prog` in Algorithm 1 line 10.

The main body of the algorithm is a loop that assembles the operations in the order of the rank numbers in *seq*, i.e., Algorithm 2 line 7 produces the next operation *op* to assemble. Lines 8 - 10 compute the membrane l , where *op* should be located, and the current membranes S where *op*'s locus belongs to; S' is a map, mapping from the current qubits in the locus to the membranes, if the membranes are different from l .

For each qubit q in S' , we insert a communication pair (`insert_send` and `insert_rev`) across the membrane $S'(q)$ and l . During the procedure, we need to generate a fresh variable for representing

the qubit renaming after the receiving operation in l . We use a map `name`, mapping from qubits to their aliasing variables, to perform such a record. In generating operations in `insert_send`, `insert_rev`, and `place`, we might need to refer to qubits' aliasing variable names in `name`, since the qubit names might change after receiving a qubit in a different membrane. Line 15 places the operation `op` in l .

Fitness Evaluation. After the distributed candidate P is generated, we evaluate it using the fitness function $\text{fit}(P)$ (Algorithm 1 line 13), which outputs a score indicating the program's satisfaction with respect to the defined criteria. This paper defines satisfaction as the balance between the maximal qubit usage in a membrane and the total number of inserted communication pairs across all membranes. The fitness function can certainly be generalized to different fitness functions, which will be the focus of our future study.

Let $\text{max_mem}(P)$ compute the maximal qubit usage in every membrane in a candidate P and $\text{com_num}(P)$ count the communication pairs inserted in P . We set up the fitness function $\text{fit}(P)$ to bound the number of qubits used in each membrane to N and to minimize the number of communication pairs; this fitness function is implemented as follows.

$$\text{fit}(P) \triangleq \text{if } \text{max_mem}(P) \leq N \text{ then } \text{com_num}(P) \text{ else } \infty$$

The fitness function is set to be the number of communication pairs if the maximal qubit usage for each membrane is less than a threshold; otherwise, we set the fitness to be ∞ . We set the final output program to be the minimization of the fitness evaluation of P , i.e., $\min_P(\text{fit}(P))$.

The computation of $\text{max_mem}(P)$ is based on the qubit usage in the execution flow of each membrane. The maximal qubit usage calculation is based on the qubit usage at a given time during the execution in a membrane, e.g., the qubit usage in the membrane l in Theorem 6.3 is only 1 at a given time, because the membrane l creates a qubit with a single Hadamard application and works on another qubit after the first qubit is sent out to the membrane u . The observation is that it is not hard to have qubits in NISQ quantum computers, but it is challenging to have many qubits work together at the same time on a task in a single-threaded machine, especially when these qubits are entangled with each other. The bottleneck is reducing qubit usage within a given time frame across all membranes. Therefore, using quantum channels to relocate qubits from one membrane to the other is the solution to reduce the bottleneck. $\text{com_num}(P)$ counts the communication pairs, i.e., the quantum sending and receiving operations, in the whole program. We can count the number of fresh quantum channel names generated by the program during execution.

In Section 4.1, `AUTODisQ` generates many candidate distributed Shor's algorithm programs as we apply the fitness function to evaluate each of the candidates, via the setting of N to be 4. Eventually, Theorem 6.3 is produced because it has the fewest generated communication pairs.

We show the soundness of the `AUTODisQ` algorithm below, where two quantum states Φ_1 and Φ_2 are similar, as $\Phi_1 \sim \Phi_2$, if and only if they are equivalent regardless of the locus membrane location information.

THEOREM 4.3 (AUTODisQ SOUNDNESS). Let $\llbracket R \rrbracket_I$ be a sequential quantum program, partitioning $\llbracket R \rrbracket_I$ into a set of membranes S via `AUTODisQ`, with $|S| \geq 1$, results in a program Q , for any quantum state Φ and Φ_1 , with $(\Phi, P) \rightarrow^* (\Phi_1, \emptyset)$, there exists Φ_2 , $(\Phi, Q) \rightarrow^* (\Phi_2, \emptyset)$ and $\Phi_1 \sim \Phi_2$.

PROOF. The proof is in `Rocq` and performs induction on $\llbracket R \rrbracket_I$, and the membrane set S . $\square$

5 Evaluation

This section presents an experimental evaluation of `AUTODisQ`. We first define our benchmark suite of quantum programs, including the GHZ, QFT, ripple-carry adder (RC-adder), QFT-adder, Shor's,

Quantum Programs	Total Qubits for 8 QBs / 16 QBs	Operation Num.
GHZ	8/16	17 / 33
RC-adder	18/34	34 / 66
QFT	8/16	44/152
QFT-adder	16/32	26/50
Shor's Algo.	16/32	35 / 67
Hamming Weight State Prep.	12/24	28 / 56
Amplitude Estimation	16/32	36 / 68
Discrete Log	24/48	50/98

Table 1. Quantum program dataset; We list the qubits and operation numbers for the case of each qubit register having 8 and 16 qubits (QBs).

Programs	2 Membranes		3 Membranes		5 Membranes	
	8QB T / MX	16QB t / MX	8QB T / MX	16QB T / MX	8QB T / MX	16QB T / MX
GHZ	< 2s / 5	<2s / 9	< 2s / 4	4s / 7	< 2s / 3	11s / 5
RC-adder	< 2s / 10	5s / 18	< 2s / 7	6s / 13	3s / 6	31s / 9
QFT	4s / 5	76s / 9	6s / 4	153 / 8	18s / 4	573s / 5
QFT-adder	< 2s / 9	<2s / 17	< 2s / 9	5s / 17	< 2s / 9	27s / 17
Shor's Algo.	< 2s / 9	10s / 17	4s / 9	24s / 17	7s / 9	132s / 17
Ham-Weight.	< 2s / 9	9s / 17	< 2s / 5	17s / 9	< 2s / 6	73s / 10
Amp. Est.	3s / 9	50s / 17	5s / 9	121s / 17	12 / 9	431s / 17
Discrete Log	8s / 13	56s / 25	11s / 9	133s / 17	17s / 9	488s / 17

Table 2. AutoDisQ Running Result. We measure the time (T) and max qubit usage in a membrane (MX), including photonic qubits for communication pairs; time is recorded in seconds (s). We show the result for partitioning programs into 2, 3, and 5 membranes.

Hamming-weight state preparation, amplitude estimation, Discrete Log algorithms, to assess how AUTODisQ can be used to build distributed quantum programs effectively. We then present our results, structured to answer the following research questions:

- RQ1: Effectiveness and Scalability: Is AUTODisQ effective and scalable enough to generate a distributed quantum program, ensuring that fitness is met?
- RQ2: Applicability: Is AUTODisQ able to be used to ensure other fitness criteria?

Implementation. We implement AUTODisQ in Rocq with its correctness verification, and extract the Rocq code to Ocaml. We then run the Ocaml version on the seven programs with different qubit counts (8 and 16) and membrane location settings (2, 3, or 5). The program dataset is given in Table 1.

Experimental Setup. We perform our evaluation of the AUTODisQ framework on an Ubuntu computer, which has 8-core 13-gen i9 Intel processors and 16 GiB DDR5 memory.

5.1 Effectiveness and Scalability

We first conduct an experiment on the effectiveness and scalability of using AUTODisQ to partition a sequential program into its distributed version. Results are documented in Table 2. We distribute 8 different quantum programs, having registers of 8 and 16 qubits, to distributed versions of 2, 3, and 5 membranes.

The table shows that all programs have been successfully rewritten as distributed versions. Specifically, the rewrites of 8-qubit programs can mostly be finished within two seconds, with a few exceptions. All of them are finished within half a minute. When rewriting a medium-sized

Programs	2 Membranes		3 Membranes		5 Membranes	
	8QB MX / C.	16QB MX / C.	8QB MX / C.	16QB MX / C.	8QB MX / C.	16QB MX / C.
GHZ	5 / 1	9 / 1	4 / 2	7 / 2	3 / 4	5 / 4
RC-adder	10 / 2	18 / 2	7 / 4	13 / 4	5 / 8	8 / 8
QFT	5 / 4	9 / 8	4 / 4	7 / 8	3 / 4	5 / 4
QFT-adder	9 / 16	17 / 32	9 / 16	17 / 32	9 / 16	17 / 32
Shor's Algo.	9 / 16	17 / 32	9 / 16	17 / 32	9 / 16	17 / 32
Ham-Weight.	9 / 16	17 / 32	5 / 16	9 / 32	5 / 16	9 / 32
Amp. Est.	9 / 16	17 / 32	9 / 16	17 / 32	9 / 16	17 / 32
Discrete Log	13 / 24	25 / 48	9 / 24	17 / 48	9 / 24	17 / 48

Table 3. Expert hand-crafted baselines for partitioning into 2, 3, and 5 membranes. We report the maximal qubit usage (MX) in a membrane, including photonic qubits used by communication pairs, together with the total number of communication pairs (C.).

program with 16 qubit registers, efficiency depends on the number of available membranes and the program's structure. All the programs can be successfully rewritten in 3 minutes, given 2 and 3 membranes. It is a little time-consuming to rewrite some programs, with 5 membranes available. For example, the rewrites of the QFT and Discrete Log algorithms require 10 minutes.

As shown in Table 2, the executions of AUTODisQ can help reduce the qubit usage in a single membrane. Distributing a program across 2 membranes can yield an average 40% reduction in qubit usage, whereas distributing it across 3 or 5 membranes yields more than 50% reduction, including the qubits added for communication pairs.

Depending on the programs, adding more available membranes for program partitioning might or might not reduce qubit usage per membrane. Some programs support operation abstraction, including the ripple-carry adder (RC-adder), QFT-adder, Shor's algorithm, Hamming weight state preparation, and the discrete log algorithm. For example, in Shor's algorithm, we abstract a controlled modular multiplication circuit as a single operation. Such abstraction can improve the AUTODisQ efficiency, but it means that there is a lower bound of qubit usage in a membrane, e.g., 9 and 17 qubits for the 8 and 16 qubit Shor's algorithm, respectively. In such a case, we adapt a hierarchical program partitioning strategy; see Section 6.2. The GHZ and QFT programs do not use any operation abstraction; therefore, the minimal single-membrane qubit usage will be the maximum gate qubit number (the maximum qubit number for a single gate) plus the number of existing communication pairs at a time.

The success of distributing a sequential program should be based on its comparison with an expert hand-crafted distributed program; such programs are listed in Table 3, where we show, for each program in Table 1, the maximum qubit usage per membrane and minimal inserted communication pairs. Distributing a sequential program might result in a naive solution. For example, when researchers started designing the distributed Shor's algorithm, they tried to place the operations (Figure 5) in the l and u membranes in a single QPU [Yimsiriwattana and Lomonaco Jr. 2004], which results in a gate-saving circuit without saving qubit usage in each QPU. Our distributed Shor's algorithm in Theorem 6.3 can be viewed as an expert-crafted program, as our researchers spent a long time finding the right partition. Section A demonstrates an example of a naive and expert hand-crafted program for the ripple-carry adder. Compared with the expert-crafted program, the results generated by AUTODisQ in Table 2 are close enough. This demonstrates the effectiveness of AUTODisQ.

Programs	Leaning Saving Operations		Leaning Saving Communications	
	Max Ops	Total Comm.	Max Ops	Total Comm.
GHZ	7	7	10	4
RC-adder	14	14	20	8
QFT	31	10	58	4
QFT-adder	10	43	23	32
Shor's Algo.	14	56	32	32
Ham-Weight.	12	46	28	32
Amp. Est.	14	51	33	33
Discrete Log	20	74	49	49

Table 4. Result (16 QBs) for the two scenarios of saving operations or comm. pairs. We record max operation usage in a membrane (Max Ops) vs. the total number of communication pairs being inserted (Total Comm.).

5.2 Taking into Account Different Qubit Performance

In addition to optimizing qubit usage, one can also use `AUTODisQ` to optimize gate usage within each membrane, as gate depth in a QPU is critical for the execution of a quantum program nowadays. On the other hand, in implementing a distributed quantum system, the communication pairs via quantum channels are implemented via photonic links, while normal qubits in a single-threaded quantum computer are typically not photon-based³. The latter has a gate execution time 60 times faster than the former.

The evaluation in Section 5.1 only considers qubit usage, whereas our fitness function can be configured to optimize gate usage within each membrane and minimize the use of photonic links. We conduct an experiment on the two different fitness while ensuring minimizing qubit usage in a membrane, 1) minimizing the maximal gate usage in a membrane, and 2) minimizing the photonic link usage. The result is documented in Table 4. Here, we only conduct the experiment for partitioning 16-qubit programs into 5 membranes. The left-hand two columns record the results of trying to save gates, while the right-hand column records the results of trying to save photonic links.

Saving gates results a roughly uniformed distribution of program operations in different membranes, in such a case, we need to add more photonic links to permit relocation of qubits, compared to the situation of saving communication pairs on the right. Sometimes, it might not be worth it. If we choose to save photonic links, in many cases, some membranes contain many operations, while others contain none. The output shows a match between the inserted photonic link numbers and the expert hand-crafted results in Table 3. This shows the effectiveness of `AUTODisQ` in guaranteeing properties beyond maximum qubit usage. The choice of fitness depends on the target machines, and `AUTODisQ` allows different fitnesses for users.

6 Discussion with Case Studies

This section discusses additional single-threaded parallelism that can be performed in `AUTODisQ` as well as the hierarchical program partitioning strategy.

6.1 Discussion: Single-threaded Software Parallelism

After partitioning a quantum program into different membranes, sometimes, the circuit depth might still be too large in a single membrane, such as the situation in membrane u in Shor's algorithm in Theorem 6.3. A possible solution is to apply single-threaded parallelism to parallelize single-threaded processes into parallelized processes (software parallelism). Such a mechanism

³IBM uses superconducting qubits, while other companies use trapped-ion or neutral atom qubits.

Algorithm 3 Automated Parallelization Algorithm

```

1: Input:  $hb_l, seq_l$ 
2: Output:  $\bar{R}$ 
3:  $hb' \leftarrow \text{opt\_hp}(hb_l, seq_l)$ 
4:  $S \leftarrow \text{scc}(hb')$ 
5:  $\bar{R} \leftarrow \emptyset$ 
6: while  $\bar{s} \in S$  do
7:    $R \leftarrow \text{gen\_ops}(\bar{s})$ 
8:    $\bar{R} \leftarrow \bar{R}, R$ 
9: end while
10: Return  $\bar{R}$ 

```

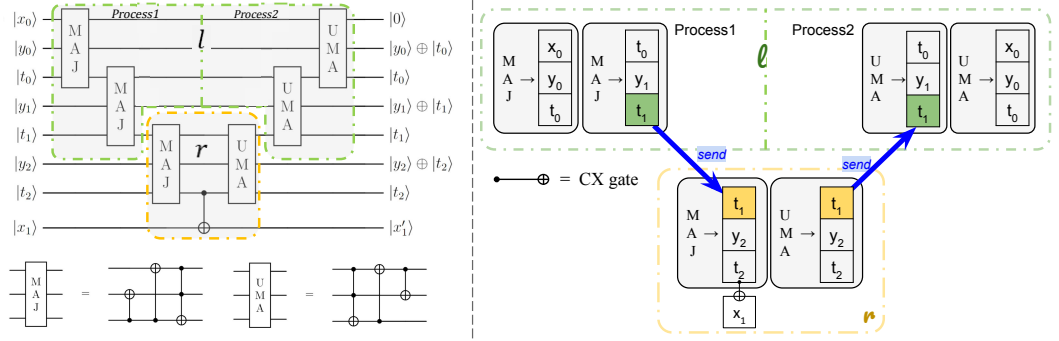


Fig. 13. Ripple-carry adder. (left): sequential ver.; (right): distributed ver.; x'_1 : overflow bit.

exists in the current IBM quantum machine [AbuGhanem 2025]. In AUTODISQ, to parallelize a single-threaded membrane, we extend our syntax to allow multiple processes in a membrane (syntax and semantics in Section B).

Algorithm 3 shows the automated parallelization algorithm. The algorithm can simply be summarized as follows: given a hb relation for the process in a membrane, we find the strongest connected components in hb . We then parallelize the operations within each component into a single process in the membrane. In the membrane l in Theorem 6.3, we can sequentialize each Hadamard operation $x[j] \leftarrow H$ and message sending operation $c!x[j].0$, for $j \in [0, 3)$, so it results in three parallelized processes as follows.

$$x[0] \leftarrow H.c_0!x[0].0, x[1] \leftarrow H.c_1!x[1].0, x[2] \leftarrow H.c_2!x[2].0$$

6.1.1 Distributed Quantum Ripple-carry Adders. We demonstrate the parallelism via an example of distributing a quantum ripple-carry adder. Quantum oracle circuits are reversible and used as subroutines in many quantum algorithms; they usually perform the quantum version of some classical computations, e.g., the oracle component in Shor's algorithm is a quantum version of a modulo-multiplication circuit. They are usually the most resource-intensive component in a quantum circuit [Li et al. 2022] and can be implemented as arithmetic operations using quantum addition circuits. Distributing the operations of oracle circuits across remote machines can mitigate the entanglement resource requirements at a single location. Here, we show the example of rewriting

a quantum ripple-carry adder [Cuccaro et al. 2004] into a distributed one and the distributed QFT-based adders in Section 6.2.1.

Figure 13 (left) shows the sequential circuit of a three-qubit ripple-carry adder, adding the value of a three-qubit array t to the value stored in the three-qubit array y . The two-qubit array x stores extra carry qubits, one for the initial carry and the other for an overflow indicator.

$$x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{MAJ} . x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{UMA} . 0$$

A quantum ripple-carry adder is constructed by a series of MAJ operations followed by a series of UMA operations, each of which has a diagram on the left side of Figure 13 (left). To understand the effect of the MAJ and UMA pairs, we show the application of such a pair to qubits $x[0]$, $y[0]$, and $t[0]$ above. Here, $x[0]$ is a carry flag for lower significant bits, and $y[0]$ and $t[0]$ are the two bits to add. The application of the MAJ operation adds $t[0]$ to $y[0]$, computes the carry flag for the next significant position, and stores the bit in $t[0]$. The application of the UMA operation reverses the computation in $x[0]$ and $t[0]$ back to their initial bits, but computes the additional result of adding $x[0]$, $y[0]$, and $t[0]$, stored in $y[0]$. As shown in Figure 13, we arrange the MAJ and UMA sequences in the pattern that every MAJ and UMA pair is placed to connect a carry bit and two bits in the same significant position of arrays y and t . The CX gate in the middle of the circuit produces the overflow flag stored in $x[1]$. We define these steps in `DisQ` as follows.

We demonstrate the use of `AutoDisQ` to rewrite an 3-qubit (3 qubit each for y and t register) sequential ripple-carry adder into a distributed version of two membranes l and r , where the number n is set to be 3 and the sequential adder utilizes the three loci, $x[0, 2]$, $y[0, 3]$, and $t[0, 3]$. The goal is to ensure that the maximum qubit usage is ≤ 6 , while minimizing the number of inserted communication pairs.

Based on the sequential program in Figure 13 (left), we generate an operation list `os`, by labeling the seven operations from 0 to 6, and a `fib` relation, having the edges $\{j \rightarrow j+1\}$ for all $j \in [0, 6]$. When generating the program relations, the sequenced-before relation is the same as `fib`, as `seq = fib`. The generation of membrane assignments is a random choice of assigning the two membrane labels, l and r , to the seven operations, as well as the 8 initial qubits.

Based on the generated program relations, we generate distributed programs for the sequential adder and evaluate them based on the fitness of setting maximal qubit usage to 6 and minimizing the communication pair usage. Eventually, we produce the optimized program in Figure 13 and the program below.

Example 6.1 (Distributed Ripple-Carry Adder). The following program represents a 3-qubit distributed ripple-carry addition circuit and has two membranes l and r . Qubits $x[0]$, $y[0, 2]$, and $t[0, 2]$ belong to membrane l , and qubits $x[1]$, $y[2]$, and $t[2]$ belong to membrane r . Qubit arrays y and t are the input qubits storing two 3-qubit bitstrings as numbers, y stores the final output of adding the two numbers, and $x[0]$ is an ancilla initial carry qubit, $x[1]$ stores the overflow bit.

$$\begin{aligned} \{ & x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{MAJ} . t[0] \sqcup y[1] \sqcup t[1] \leftarrow \text{MAJ} . c!t[1] . 0 . \\ & c'?(w) . t[0] \sqcup y[1] \sqcup w \leftarrow \text{UMA} . x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{UMA} . 0 \} _l , \\ \{ & c'?(w) . w \sqcup y[2] \sqcup t[2] \leftarrow \text{MAJ} . t[2] \sqcup x[1] \leftarrow \text{CX} . c[0] \sqcup y[2] \sqcup t[2] \leftarrow \text{UMA} . c'!w . 0 \} _r \end{aligned}$$

In evaluating the program, we discover two key patterns, which can generalize the optimization in `AutoDisQ`. First, quantum programs are reversible with many symmetric program patterns, such as the MAJ and UMA pairs in Figure 13, and symmetric program operations are typically applied on the same qubits. In many cases, it is optimal to assume that a large set of qubits is fixed in a membrane while only transmitting a small set of qubits. Thus, there will be an even number of inserted communication pairs, where we transmit the small set of qubits in a membrane and then transmit the qubit information back to the original membrane, because we want to apply

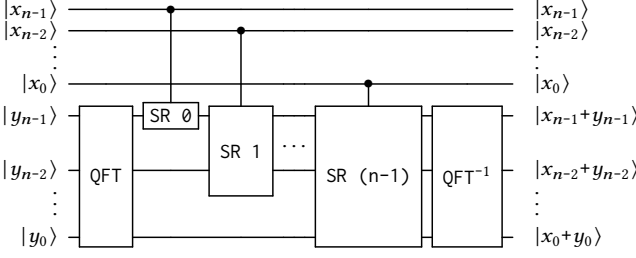


Fig. 14. Quantum QFT-Based Adder Circuit

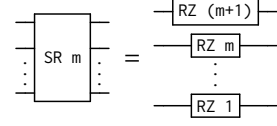


Fig. 15. SR unfolds to RZ.

both an operation and its symmetric one to the same group of qubits. For example, we apply an MAJ operation to $t[0]$, $y[0]$, and $t[1]$ in the membrane l , transmit $t[1]$ to membrane r for other operations, and then transmit it back to the membrane l to apply the symmetric UMA operation to $t[0]$, $y[0]$, and $t[1]$.

Second, we might apply additional optimization based on parallelism. Here, we can view communication pairs as the program locations for parallelization. In Figure 13 (right), the inserted receiving operation ($c'?(w)$) is synchronized with the last sending operation in the membrane r ; therefore, we can parallelize the generated process in the membrane l into two processes, i.e., we further concurrently execute the two MAJs and UMAs, respectively, through two different processes in l . In Theorem 6.1, the process in the membrane l can be partitioned into two processes as follows.

$$\begin{aligned} x[0] \sqcup y[0] \sqcup t[0] &\leftarrow \text{MAJ}.t[0] \sqcup y[1] \sqcup t[1] \leftarrow \text{MAJ}.c!t[1].0, \\ c'?(w).t[0] \sqcup y[1] \sqcup w &\leftarrow \text{UMA}.x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{UMA}.0 \end{aligned}$$

6.2 Discussion: Hierarchical Auto Program Partitioning

As shown above, the program partitioning of some algorithms includes abstract gates. In Shor's algorithm, we use abstract operations to represent the modular multiplication, instead of implementing its complete quantum circuit, to avoid AUTODisQ to partition the modular multiplication circuits into different distributed membranes. This permits AUTODisQ to avoid dealing with a large number of gates. By abstracting the modular multiplication circuits, AUTODisQ deals with a version of Shor's algorithm with 129 gates, while a typical 64 modulo multiplication circuit might contain more than two million gates.

To perform auto-distribution on the whole circuit, e.g., the whole Shor's algorithm circuit, we employ a Hierarchical approach, where we first apply AUTODisQ for the top-level algorithm, and continuously apply it to auto-distribute the sub-components being abstracted away.

6.2.1 Distributed QFT Adder: an Example of Hierarchical Partitioning. Here, we provide an example of using AUTODisQ with the hierarchical partitioning strategy for a QFT-based adder. We first abstract QFT and SR gates. Once we successfully partition the adder, we further distribute a single QFT into different membranes.

A QFT-based adder (Figure 14) performs addition differently than a ripple-carry adder. It usually comes with two qubit arrays x and y , tries to sum the x bits into the y array, by first transforming x 's qubits to QFT-basis and performing addition in the basis, i.e., instead of performing bit arithmetic in a ripple-carry adder, it records addition results via phase rotations. The final inverse QFT operation QFT^{-1} transforms the addition result in the qubit phase back to basis vectors.

To apply AUTODisQ on the n -qubit sequential QFT-adder into two membranes l and r , with the requirement of $\leq n + 2$ maximal qubit usage in each membrane, we generate an operation list os to all the $n + 2$ operations and a fib relation, and then generate the program relations including the seq relation and membrane assignments. Here, the sequenced-before relation is the same as fib , as

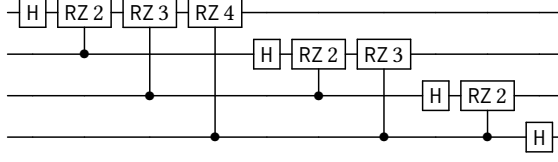


Fig. 16. Example QFT over 4 qubits.

$seq = hb$, and the generation of membrane assignments is a random choice of assigning the two membrane labels, l and r , to the operations and qubits.

By generating distributed quantum programs based on the above relations and evaluating them based on the fitness of setting maximal qubit usage to $n + 2$ and minimizing the communication pair usage, we produce the optimized program below.

Example 6.2 (Distributed QFT Adder). Membrane l holds qubit array x and membrane r takes care of qubit array y , and they share two n -qubit quantum channels c and c' . C-SR(j) is the controlled SR operation, where $x[j] \boxplus y[0, n] \leftarrow \text{C-SR}(j)$ means controlling over $x[j]$ on applying SR to the $y[0, n]$ range.

Recursive Combinator:

$$Rec(j, n, f) = \text{if } (j = n) \ 0 \ \text{else } f(j) . Rec(j+1)$$

Process Definitions:

$$Se(j) = c!x[j] . c'? \langle w \rangle \quad SeR(n) = Rec(0, n, Se)$$

$$Re(j) = c? \langle w \rangle . w \boxplus y[0, n] \leftarrow \text{C-SR}(j) . c'!w \quad ReR(n) = Rec(0, n, Re)$$

Membrane Definition:

$$\llbracket SeR(n) \rrbracket_l, \llbracket y[0, n] \leftarrow \text{QFT} . ReR(n) . y[0, n] \leftarrow \text{QFT}^{-1} . 0 \rrbracket_r$$

AUTODisQ produces a distributed program similar to the above, where we essentially place all operations and the qubit array $y[0, n]$ in the membrane r , where the initial qubits $x[0, n]$ are in the membrane l . In this case, each qubit $x[j]$ in the locus $x[0, n]$ is transmitted to the membrane r one by one, with the application of the control SR gate in the membrane r . To ensure the maximal qubit usage in the membrane r , we still need to transmit $x[j]$ back to the membrane l . In each of the above steps, we create two quantum channels (c and c'), as in the process Se above. In the j -th iteration, the membrane l sends $x[j]$ to the membrane r via the channel c , the membrane r receives the information in the qubit $x[j]$, stored as $\langle w \rangle_r$, and applies a C-SR operation that controls over the qubit $\langle w \rangle_r$ on applying SR operation on the y qubit array. Then, we teleport w back to membrane l via another single-qubit quantum channel in c' . After all qubits in the array $x[0, n]$ have been applied C-SR gate, we apply an inversed QFT gate to transform the addition result in y 's phase back to its basis vectors. In each iteration, two communication pairs are consumed, so the minimized communication pair consumption is $2n$.

The QFT operation in the QFT-adder example above is an abstract operation. One can continue partitioning the QFT using AUTODisQ by implementing the QFT circuit shown in Figure 16. Here, we try to partition the 4-qubit QFT into two membranes l and r by using AUTODisQ. The result of the distributed program is listed below. In this circuit, we first send the qubits $x[2]$ and $x[3]$ from the membrane r to l and apply several controlled RZ gates. We then send the qubits back to the membrane r to perform the latter operations on $x[2]$ and $x[3]$.

Example 6.3 (Distributed QFT Circuit). We have the membrane l holding qubit range $x[0, 2)$ and the membrane r holding qubit range $x[2, 4)$. They share qubit channels c_1, c_2, c'_1 , and c'_2 . C-RZ(j) is the controlled RZ operation, where $x[j] \boxplus y[k] \leftarrow \text{C-RZ}(t)$ means controlling over $x[j]$ on applying RZ to $y[k]$ for the rotation t .

Process Definitions:

$$Le1 = x[0] \leftarrow H.x[1] \sqcup x[0] \leftarrow C-RZ(2) \cdot c_1?(w_1) \cdot w_1 \sqcup x[0] \leftarrow C-RZ(3) \cdot c_2?(w_2) \cdot w_2 \sqcup x[0] \leftarrow C-RZ(4)$$

$$Le2 = x[1] \leftarrow H.w_1 \sqcup x[1] \leftarrow C-RZ(2) \cdot w_2 \sqcup x[1] \leftarrow C-RZ(3) \cdot c'_1!w_1 \cdot c'_2!w_2$$

$$Re = c_1!x[2] \cdot c_2!x[3] \cdot c'_1?(w'_1) \cdot x[2] \leftarrow H.c'_2?(w'_2) \cdot w'_2 \sqcup w'_1 \leftarrow C-RZ(2) \cdot w'_2 \leftarrow H$$

Membrane Definition:

$$\{\{Le1.Le2.0\}_l, \{Re\}_r\}$$

7 Related Work

Many previous studies inspire the AUTODisQ development.

Distributed Quantum Computation Works. Building a quantum internet and distributed systems has long been a dream for many researchers, with extensive theoretical and implementation work. For example, Beals *et al.* [Beals et al. 2013] presented a theory of performing distributed quantum computation via quantum random access memory. Buhrman and Rohrig [Buhrman and Röhrig 2003] examined the development of distributed quantum computing algorithms, and Cuomo *et al.* [Cuomo et al. 2023] proposed an optimized compiler for distributed quantum computing. Panagiotis *et al.* [Promponas et al. 2024] developed a compiler for distributed quantum computation, which takes in a distributed quantum program and compiles it to different QPUs. Sundaram and Gupta [Sundaram and Gupta 2023] proposed a distributed quantum computation hardware model via quantum teleportation [Bennett et al. 1993; Rigolin 2005]. Sundaram *et al.* [G Sundaram et al. 2021; Sundaram et al. 2025, 2022] proposed an algorithm to partition a quantum circuit into small pieces, each of which can be executed in parallel. Although the algorithm can be used for a distributed quantum program, the work did not propose a framework to automatically distribute a sequential quantum program into a distributed one with optimized resource allocation. LarQucut [Dou et al. 2025] proposed an algorithm based on heuristics to cut large circuits into smaller ones that can be used in different QPUs in a distributed quantum computing network. Compared to AUTODisQ, the computation does not automatically satisfy the constraint, and there is no correctness guarantee of the algorithm.

Concurrent Quantum Frameworks. Many previous works studied the possibilities of quantum concurrency, in which a quantum program can be partitioned and run in a multi-threaded environment. Ying and Feng [Ying and Feng 2009] proposed an algebraic logical system to help partition a sequential quantum program into sub-components that can be executed in parallel, providing properties for constructing a distributed quantum system. Partitioned components might share qubits, which indicates that the proposed partitions represent a concurrent system. Feng *et al.* [Feng et al. 2022] proposed a proof system for concurrent quantum programs based on quantum Hoare logic [Ying 2012]. Ying *et al.* [Ying et al. 2018, 2022] carefully design a quantum concurrent proof system by combining the above two works to permit the concurrent quantum program verification. Zhang and Ying [Zhang and Ying 2024] extended the quantum concurrent proof system with the consideration of atomicity.

Eisert *et al.* [Eisert et al. 2000] showed the resource estimation of implementing a non-local gate theoretically, without investigating how long-distance entanglement can be established. Ardeshtir-Larijani *et al.* [Ardeshtir-Larijani et al. 2014] developed equivalence checkers for concurrent quantum programs, while Ardeshtir-Larijani *et al.* [Ardeshtir-Larijani et al. 2013] developed an equivalence checker for quantum networking protocols.

These works primarily focus on reasoning about quantum concurrent behaviors, assuming the existence of distributed quantum programs and treating interactions among distributed components as concurrent processes. In contrast, AUTODisQ identifies methods for constructing distributed quantum systems and analyzes how a sequential program can be transformed into a distributed

one. The proof systems proposed by Ying *et al.* [Ying et al. 2018, 2022] assume communication via classical channels (unlike in our case, where we also consider quantum teleportation as a means of communication). Additionally, Zhang and Ying [Zhang and Ying 2024] address atomicity, exploring the consequences when two concurrent quantum programs manipulate the same qubits.

Quantum Process Algebra. Tafliovich and Hehner [Tafliovich and Hehner 2009] proposed a framework to specify quantum network protocols. The AUTODisQ language design was inspired by several existing quantum process calculi: Disq [Chang et al. 2025], qCCS [Feng et al. 2012; Qin et al. 2020; Ying et al. 2009], Communicating Quantum Processes (CQP) [Gay and Nagarajan 2005], quantum model checker (MQC) [Davidson et al. 2012; Gay et al. 2013], QPAlg [Jorrand and Lalire 2004], and eQPAlg [Haider and Kazmi 2020]. These process calculi are developed to describe quantum networking and security protocols.

Traditional Process Algebra. Communicating Sequential Processes (CSP) [Hoare 1985] and Π -calculus [Milner et al. 1992] are process calculi suitable for defining concurrent systems based on the message-passing model. Several bisimulation and trace-refinement protocol verification methodologies exist for CSP and the Π -calculus [Gibson-Robinson et al. 2014; Ltd. 2010; Sangiorgi 1993]. The Chemical Abstract Machine [Berry and Boudol 1992] is the inspiration of the AUTODisQ language.

Single-threaded Quantum Circuit Programming Languages. There are many single-location quantum circuit-based language developments. Q# [Svore et al. 2018], Quilc [Smith et al. 2020], Scaffold [JavadiAbhari et al. 2015], Project Q [Steiger et al. 2018], Cirq [Google Quantum AI 2019], Qiskit [Aleksandrowicz et al. 2019] are industrial quantum circuit languages. There are formally verifying quantum circuit programs, including Qwire [Rand 2018], SQIR [Hietala et al. 2021], and QBricks [Chareton et al. 2021], quantum Hoare logic and its subsequent works [Liu et al. 2019; Ying 2012; Zhou et al. 2023], Qafny [Cheng et al. 2025; Li et al. 2024]. These tools have been used to verify a range of quantum algorithms, from Grover’s search to quantum phase estimation. There are works verifying quantum circuit optimizations (e.g., voqc [Hietala et al. 2023], CertiQ [Shi et al. 2019]), as well as verifying quantum circuit compilation procedures, including ReVerC [Amy et al. 2017] and ReQWIRE [Rand et al. 2018]. There are single-location circuit-based equivalence checkers [Chen et al. 2022; Peham et al. 2022; Shi et al. 2020; Sun and Wei 2022; Wang et al. 2022, 2021; Yamashita and Markov 2010] for verifying quantum compiler optimizations.

8 Conclusion and Future Work

We present AUTODisQ, an automated system that generates distributed quantum programs from a sequential one with correctness guarantees. In AUTODisQ, users can automatically rewrite a sequential quantum program to a distributed one that can be executed in a remote distributed quantum system. The benefit of such rewrites is to mitigate the restriction on the size of entangled qubits in single-location quantum computers; i.e., a distributed quantum program can be executed across multiple quantum processors, enabling larger entanglement.

We also demonstrate AUTODisQ’s power by applying it to rewrite 8 programs, with different setups, to distributed quantum programs. AUTODisQ’s result programs have a similar resource consumption as the human expert solutions. In addition, we demonstrate how to properly set up the fitness functions so that one can not only guarantee minimizing the qubit usage in each membrane, but also guarantee other properties, such as minimizing gate usage or communication pair insertions. In short, AUTODisQ can quickly rewrite a sequential quantum program to a distributed version with the satisfaction of multi-objective criteria. We also discuss an effective way to use AUTODisQ to properly distribute a large program through hierarchical program partitioning, as well as to enable software parallelism using the AUTODisQ framework.

9 Data-Availability Statement

The DSL library and the case studies in the paper are available on Zenodo: <https://doi.org/10.5281/zenodo.19059463>. The artifact is produced in the described experimental setting and includes the theorem proof and the experiments described in the paper.

References

- Muhammad AbuGhanem. 2025. IBM quantum computers: evolution, performance, and future directions. *The Journal of Supercomputing* 81, 5 (April 2025). doi:10.1007/s11227-025-07047-7
- Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, Francisco Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, Jerry M. Chow, Antonio D. Córcoles-Gonzales, Abigail J. Cross, Andrew Cross, Juan Cruz-Benito, Chris Culver, Salvador De La Puente González, Enrique De La Torre, Delton Ding, Eugene Dumitrescu, Ivan Duran, Pieter Eendebak, Mark Everitt, Ismael Faro Sertage, Albert Frisch, Andreas Fuhrer, Jay Gambetta, Borja Godoy Gago, Juan Gomez-Mosquera, Donny Greenberg, Ikko Hamamura, Vojtech Havlicek, Joe Hellmers, Łukasz Herok, Hiroshi Horii, Shaohan Hu, Takashi Imamichi, Toshinari Itoko, Ali Javadi-Abhari, Naoki Kanazawa, Anton Karazeev, Kevin Krsulich, Peng Liu, Yang Luh, Yunho Maeng, Manoel Marques, Francisco Jose Martín-Fernández, Douglas T. McClure, David McKay, Srujan Meesala, Antonio Mezzacapo, Nikolaj Moll, Diego Moreda Rodríguez, Giacomo Nannicini, Paul Nation, Pauline Ollitrault, Lee James O’Riordan, Hanhee Paik, Jesús Pérez, Anna Phan, Marco Pistoia, Viktor Prutyanov, Max Reuter, Julia Rice, Abdón Rodríguez Davila, Raymond Harry Putra Rudy, Mingi Ryu, Ninad Sathaye, Chris Schnabel, Eddie Schoute, Kanav Setia, Yunong Shi, Adenilton Silva, Yukio Siraichi, Seyon Sivarajah, John A. Smolin, Mathias Soeken, Hitomi Takahashi, Ivano Tavernelli, Charles Taylor, Pete Taylour, Kenso Trabling, Matthew Treinish, Wes Turner, Desiree Vogt-Lee, Christophe Vuillot, Jonathan A. Wildstrom, Jessica Wilson, Erick Winston, Christopher Wood, Stephen Wood, Stefan Wörner, Ismail Yunus Akhalwaya, and Christa Zoufal. 2019. Qiskit: An open-source framework for quantum computing. doi:10.5281/zenodo.2562110
- A. Ambainis. 2004. Quantum walk algorithm for element distinctness. In *45th Annual IEEE Symposium on Foundations of Computer Science*. 22–31. doi:10.1109/FOCS.2004.54
- Matthew Amy, Martin Roetteler, and Krysta M. Svore. 2017. Verified Compilation of Space-Efficient Reversible Circuits. In *Computer Aided Verification*, Rupak Majumdar and Viktor Kunčák (Eds.). Springer International Publishing, Cham, 3–21.
- Ebrahim Ardeshtir-Larijani, Simon J. Gay, and Rajagopal Nagarajan. 2013. Equivalence Checking of Quantum Protocols. In *Tools and Algorithms for the Construction and Analysis of Systems*, Nir Piterman and Scott A. Smolka (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 478–492.
- Ebrahim Ardeshtir-Larijani, Simon J. Gay, and Rajagopal Nagarajan. 2014. Verification of Concurrent Quantum Protocols by Equivalence Checking. In *Tools and Algorithms for the Construction and Analysis of Systems*, Erika Ábrahám and Klaus Havelund (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 500–514.
- Robert Beals, Stephen Brierley, Oliver Gray, Aram W. Harrow, Samuel Kutin, Noah Linden, Dan Shepherd, and Mark Stather. 2013. Efficient distributed quantum computing. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 469, 2153 (May 2013), 20120686. doi:10.1098/rspa.2012.0686
- Stephane Beauregard. 2003. Circuit for Shor’s Algorithm Using $2n+3$ Qubits. *Quantum Info. Comput.* 3, 2 (March 2003), 175–185.
- Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. 1993. Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Phys. Rev. Lett.* 70 (Mar 1993), 1895–1899. Issue 13. doi:10.1103/PhysRevLett.70.1895
- Gérard Berry and Gérard Boudol. 1992. The chemical abstract machine. *Theoretical Computer Science* 96, 1 (1992), 217–248. doi:10.1016/0304-3975(92)90185-I
- Harry Buhrman and Hein Röhrig. 2003. Distributed Quantum Computing. 1–20. doi:10.1007/978-3-540-45138-9_1
- Marcello Caleffi, Michele Amoretti, Davide Ferrari, Daniele Cuomo, Jessica Illiano, Antonio Manzalini, and Angela Sara Cacciapuoti. 2022. Distributed Quantum Computing: a Survey. (12 2022). arXiv:2212.10609 [quant-ph]
- Le Chang, Saitej Yavvari, Rance Cleaveland, Samik Basu, Runzhou Tao, and Liyi Li. 2025. DisQ: A Model of Distributed Quantum Processors. arXiv:2407.09710 [quant-ph] <https://arxiv.org/abs/2407.09710>
- Christophe Charetton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An Automated Deductive Verification Framework for Circuit-building Quantum Programs. In *Programming Languages and Systems - 30th European Symposium on Programming, ESOP 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings (Lecture Notes in Computer Science, Vol. 12648)*, Nobuko Yoshida (Ed.). Springer, 148–177. doi:10.1007/978-3-030-72019-3_6
- Tian-Fu Chen, Jie-Hong R. Jiang, and Min-Hsiu Hsieh. 2022. Partial Equivalence Checking of Quantum Circuits. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE. doi:10.1109/qce53715.2022.00082

- Wentao Chen, Yao Lu, Shuaining Zhang, Kuan Zhang, Guan hao Huang, Mu Qiao, Xiaolu Su, Jialiang Zhang, Jing-Ning Zhang, Leonardo Banchi, M. S. Kim, and Kihwan Kim. 2023. Scalable and programmable phononic network with trapped ions. *Nature Physics* 19, 6 (01 Jun 2023), 877–883. doi:10.1038/s41567-023-01952-5
- Feifei Cheng, Sushen Vangeepuram, Henry Allard, Seyed Mohammad Reza Jafari, Alex Potanin, and Liyi Li. 2025. Embedding Quantum Program Verification into Dafny. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 379 (Oct. 2025), 27 pages. doi:10.1145/3763157
- Andrew Childs, Ben Reichardt, Robert Spalek, and Shengyu Zhang. 2007. Every NAND formula of size N can be evaluated in time $N^{1/2+o(1)}$ on a Quantum Computer. (03 2007).
- Cheng Chu, Zhenxiao Fu, Yilun Xu, Gang Huang, Hausi Muller, Fan Chen, and Lei Jiang. 2024. TITAN: A Distributed Large-Scale Trapped-Ion NISQ Computer. arXiv:2402.11021 [quant-ph] <https://arxiv.org/abs/2402.11021>
- Steven Cuccaro, Thomas Draper, Samuel Kutin, and David Moulton. 2004. A new quantum ripple-carry addition circuit. (11 2004).
- Daniele Cuomo, Marcello Caleffi, Kevin Krsulich, Filippo Tramonto, Gabriele Agliardi, Enrico Prati, and Angela Sara Cacciapuoti. 2023. Optimized Compiler for Distributed Quantum Computing. *ACM Transactions on Quantum Computing* 4, 2, Article 15 (feb 2023), 29 pages. doi:10.1145/3579367
- Tim Davidson, Simon Gay, H. Mlnarík, Rajagopal Nagarajan, and Nick Papanikolaou. 2012. Model Checking for Communicating Quantum Processes. *International Journal of Unconventional Computing* 8 (01 2012), 73–98.
- Xinglei Dou, Lei Liu, Zhuohao Wang, and Pengyu Li. 2025. LarQcut: A New Cutting and Mapping Approach for Large-sized Quantum Circuits in Distributed Quantum Computing (DQC) Environments. *ACM Trans. Archit. Code Optim.* 22, 3, Article 88 (Sept. 2025), 24 pages. doi:10.1145/3730585
- J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio. 2000. Optimal local implementation of nonlocal quantum gates. *Phys. Rev. A* 62 (Oct 2000), 052317. Issue 5. doi:10.1103/PhysRevA.62.052317
- Yuan Feng, Runyao Duan, and Mingsheng Ying. 2012. Bisimulation for Quantum Processes. *ACM Trans. Program. Lang. Syst.* 34, 4, Article 17 (dec 2012), 43 pages. doi:10.1145/2400676.2400680
- Yuan Feng, Sanjiang Li, and Mingsheng Ying. 2022. Verification of Distributed Quantum Programs. *ACM Trans. Comput. Logic* 23, 3, Article 19 (apr 2022), 40 pages. doi:10.1145/3517145
- Ranjani G Sundaram, Himanshu Gupta, and C. R. Ramakrishnan. 2021. Efficient Distribution of Quantum Circuits. In *35th International Symposium on Distributed Computing (DISC 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 209)*, Seth Gilbert (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 41:1–41:20. doi:10.4230/LIPIcs.DISC.2021.41
- Simon Gay, Rajagopal Nagarajan, and Nikolaos Papanikolaou. 2013. Specification and Verification of Quantum Protocols. *Semantic Techniques in Quantum Computation* (01 2013). doi:10.1017/CBO9781139193313.012
- Simon J. Gay and Rajagopal Nagarajan. 2005. Communicating Quantum Processes. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Long Beach, California, USA) (POPL '05). Association for Computing Machinery, New York, NY, USA, 145–157. doi:10.1145/1040305.1040318
- Thomas Gibson-Robinson, Alexandre Boulgakov Philip Armstrong, and A.W. Roscoe. 2014. FDR3 - A Modern Refinement Checker for CSP. In *TACAS*. In Press.
- Google Quantum AI. 2019. Cirq: An Open Source Framework for Programming Quantum Computers. <https://quantumai.google/cirq>
- Salman Haider and Dr. Syed Asad Raza Kazmi. 2020. An extended quantum process algebra (eQPAI_g) approach for distributed quantum systems. arXiv:2001.04249 [cs.LO]
- Thomas Häner, Martin Roetteler, and Krysta M. Svore. 2017. Factoring Using $2n + 2$ Qubits with Toffoli Based Modular Multiplication. *Quantum Info. Comput.* 17, 7–8 (June 2017), 673–684.
- Kesha Hietala, Robert Rand, Shih-Han Hung, Liyi Li, and Michael Hicks. 2021. Proving Quantum Programs Correct. In *Proceedings of the Conference on Interactive Theorem Proving (ITP)*.
- Kesha Hietala, Robert Rand, Liyi Li, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2023. A Verified Optimizer for Quantum Circuits. *ACM Trans. Program. Lang. Syst.* 45, 3, Article 18 (Sept. 2023), 35 pages. doi:10.1145/3604630
- C. A. R. Hoare. 1985. *Communicating sequential processes*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA.
- Photonic Inc, Francis Afzal, Mohsen Akhlaghi, Stefanie J. Beale, Olinka Bedroya, Kristin Bell, Laurent Bergeron, Kent Bonsma-Fisher, Polina Bychkova, Zachary M. E. Chaisson, Camille Chartrand, Chloe Clear, Adam Darcie, Adam DeAbreu, Colby DeLisle, Lesley A. Duncan, Chad Dundas Smith, John Dunn, Amir Ebrahimi, Nathan Evetts, Daker Fernandes Pinheiro, Patricio Fuentes, Tristen Georgiou, Biswarup Guha, Rafael Haenel, Daniel Higginbottom, Daniel M. Jackson, Navid Jahed, Amin Khorshidahmad, Prasoon K. Shandilya, Alexander T. K. Kurkjian, Nikolai Lauk, Nicholas R. Lee-Hone, Eric Lin, Rostyslav Litynskyy, Duncan Lock, Lisa Ma, Iain MacGilp, Evan R. MacQuarrie, Aaron Mar, Alireza Marefat Khah, Alex Matias, Evan Meyer-Scott, Cathryn P. Michaels, Juliana Motira, Narwan Kabir Noori, Egor Ospadov, Ekta Patel, Alexander Patscheider, Danny Paulson, Ariel Petruk, Adarsh L. Ravindranath, Bogdan Reznichenko, Myles Ruether, Jeremy Ruscica, Kunal Saxena, Zachary Schaller, Alex Seidlitz, John Senger, Youn Seok Lee, Orbel Sevoyan,

- Stephanie Simmons, Oney Soykal, Leea Stott, Quyen Tran, Spyros Tserkis, Ata Ulhaq, Wyatt Vine, Russ Weeks, Gary Wolfowicz, and Isao Yoneda. 2024. Distributed Quantum Computing in Silicon. arXiv:2406.01704 [quant-ph] <https://arxiv.org/abs/2406.01704>
- IonQ. 2024. *IonQ Demonstrates Remote Ion-Ion Entanglement, a Significant Milestone in Developing Networked Quantum Systems at Scale*. <https://ionq.com/news/ionq-demonstrates-remote-ion-ion-entanglement-a-significant-milestone-in>
- Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. 2015. ScaffCC: Scalable compilation and analysis of quantum programs. *Parallel Comput.* 45 (June 2015), 2–17. doi:10.1016/j.parco.2014.12.001
- Philippe Jorrand and Marie Lalire. 2004. Toward a Quantum Process Algebra. In *Proceedings of the 1st Conference on Computing Frontiers (Ischia, Italy) (CF '04)*. Association for Computing Machinery, New York, NY, USA, 111–119. doi:10.1145/977091.977108
- Leslie Lamport. 1978. Time, Clocks and the Ordering of Events in a Distributed System. *Communications of the ACM* 21, 7 (July 1978), 558–565. Reprinted in several collections, including *Distributed Computing: Concepts and Implementations*, McEntire et al., ed. IEEE Press, 1984. (July 1978), 558–565. <https://www.microsoft.com/en-us/research/publication/time-clocks-ordering-events-distributed-system/> 2000 PODC Influential Paper Award (later renamed the Edsger W. Dijkstra Prize in Distributed Computing). Also awarded an ACM SIGOPS Hall of Fame Award in 2007..
- Liyi Li, Finn Voichick, Kesha Hietala, Yuxiang Peng, Xiaodi Wu, and Michael Hicks. 2022. Verified Compilation of Quantum Oracles. In *OOPSLA 2022*. doi:10.48550/ARXIV.2112.06700
- Liyi Li, Mingwei Zhu, Rance Cleaveland, Alexander Nicolellis, Yi Lee, Le Chang, and Xiaodi Wu. 2024. Qafny: A Quantum-Program Verifier. In *38th European Conference on Object-Oriented Programming (ECOOP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 24:1–24:31. doi:10.4230/LIPIcs.ECOOP.2024.24
- Junyi Liu, Bohua Zhan, Shuling Wang, Shenggang Ying, Tao Liu, Yangjia Li, Mingsheng Ying, and Naijun Zhan. 2019. Formal Verification of Quantum Algorithms Using Quantum Hoare Logic. In *Computer Aided Verification*, Isil Dillig and Serdar Tasiran (Eds.). Springer International Publishing, Cham, 187–207.
- Formal Systems (Europe) Ltd. 2010. Failures-Divergence Refinement. FDR2 User Manual. In *FDR2 User Manual*.
- Frédéric Magniez, Miklos Santha, and Mario Szegedy. 2005. Quantum Algorithms for the Triangle Problem. In *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms (Vancouver, British Columbia) (SODA '05)*. Society for Industrial and Applied Mathematics, USA, 1109–1117.
- D. Main, P. Drmota, D. P. Nadlinger, E. M. Ainley, A. Agrawal, B. C. Nichol, R. Srinivas, G. Araneda, and D. M. Lucas. 2024. Distributed Quantum Computing across an Optical Network Link. arXiv:2407.00835 [quant-ph] <https://arxiv.org/abs/2407.00835>
- Robin Milner, Joachim Parrow, and David Walker. 1992. A calculus of mobile processes, I. *Information and Computation* 100, 1 (1992), 1–40. doi:10.1016/0890-5401(92)90008-4
- Michael A. Nielsen and Isaac L. Chuang. 2011. *Quantum Computation and Quantum Information* (10th anniversary ed.). Cambridge University Press, USA.
- Tom Peham, Lukas Burgholzer, and Robert Wille. 2022. Equivalence Checking of Quantum Circuits With the ZX-Calculus. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 12, 3 (Sept. 2022), 662–675. doi:10.1109/jetcas.2022.3202204
- Panagiotis Promponas, Akrit Mudvari, Luca Della Chiesa, Paul Polakos, Louis Samuel, and Leandros Tassioulas. 2024. Compiler for Distributed Quantum Computing: a Reinforcement Learning Approach. arXiv:2404.17077 [quant-ph] <https://arxiv.org/abs/2404.17077>
- Xudong Qin, Yuxin Deng, and Wenjie Du. 2020. Verifying Quantum Communication Protocols with Ground Bisimulation. In *Tools and Algorithms for the Construction and Analysis of Systems*, Armin Biere and David Parker (Eds.). Springer International Publishing, Cham, 21–38.
- Robert Rand. 2018. *Formally verified quantum programming*. Ph. D. Dissertation. University of Pennsylvania.
- Robert Rand, Jennifer Paykin, Dong-Ho Lee, and S. Zdancewic. 2018. ReQWIRE: Reasoning about Reversible Quantum Circuits. In *QPL*.
- Gustavo Rigolin. 2005. Quantum teleportation of an arbitrary two-qubit state and its relation to multipartite entanglement. *Physical Review A* 71, 3 (mar 2005). doi:10.1103/physreva.71.032303
- Davide Sangiorgi. 1993. A Theory of Bisimulation for the pi-Calculus. In *CONCUR (Lecture Notes in Computer Science, Vol. 715)*, Eike Best (Ed.). Springer, 127–142.
- Yunong Shi, Xupeng Li, Runzhou Tao, Ali Javadi-Abhari, Andrew W. Cross, Frederic T. Chong, and Ronghui Gu. 2019. Contract-based verification of a realistic quantum compiler. *arXiv e-prints* (Aug 2019). arXiv:1908.08963 [quant-ph]
- Yunong Shi, Runzhou Tao, Xupeng Li, Ali Javadi-Abhari, Andrew W. Cross, Frederic T. Chong, and Ronghui Gu. 2020. CertiQ: A Mostly-automated Verification of a Realistic Quantum Compiler. arXiv:1908.08963 [quant-ph]

- P.W. Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 124–134. doi:10.1109/SFCS.1994.365700
- Robert S. Smith, Eric C. Peterson, Mark G. Skilbeck, and Erik J. Davis. 2020. An Open-Source, Industrial-Strength Optimizing Compiler for Quantum Programs. arXiv:2003.13961 [quant-ph]
- Damian S. Steiger, Thomas Häner, and Matthias Troyer. 218. ProjectQ: an open source software framework for quantum computing. *Quantum* 2 (Jan. 2018), 49. doi:10.22331/q-2018-01-31-49
- Weixiao Sun and Zhaohui Wei. 2022. Equivalence checking of quantum circuits by nonlocality. *npj Quantum Information* 8, 1 (29 Nov 2022), 139. doi:10.1038/s41534-022-00652-x
- Ranjani Sundaram, Himanshu Gupta, and CR Ramakrishnan. 2025. DQC-QR: Distributing and Routing Quantum Circuits with Minimum Execution Time. *ACM Transactions on Quantum Computing* 6, 4, Article 30 (Sept. 2025), 26 pages. doi:10.1145/3757069
- Ranjani G. Sundaram and Himanshu Gupta. 2023. Distributing Quantum Circuits Using Teleportations . In *2023 IEEE International Conference on Quantum Software (QSW)*. IEEE Computer Society, Los Alamitos, CA, USA, 186–192. doi:10.1109/QSW59989.2023.00030
- Ranjani G. Sundaram, Himanshu Gupta, and C. R. Ramakrishnan. 2022. Distribution of Quantum Circuits Over General Quantum Networks . In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE Computer Society, Los Alamitos, CA, USA, 415–425. doi:10.1109/QCE53715.2022.00063
- Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (RWDSL2018)*. ACM. doi:10.1145/3183895.3183901
- Matt Swayne. 2024. Google Is Looking For Proposals to Push Boundaries in Distributed Quantum Computing. <https://thequantuminsider.com/2024/06/18/google-is-looking-for-proposals-to-push-boundaries-in-distributed-quantum-computing/>
- Anya Taflivovich and Eric C.R. Hehner. 2009. Programming with Quantum Communication. *Electronic Notes in Theoretical Computer Science* 253, 3 (2009), 99–118. doi:10.1016/j.entcs.2009.10.008 Proceedings of Seventh Workshop on Quantitative Aspects of Programming Languages (QAPL 2009).
- A.S. Tanenbaum and M. van Steen. 2017. *Distributed Systems*. CreateSpace Independent Publishing Platform. <https://books.google.com/books?id=c77GAQAACAAJ>
- Wei Tang and Margaret Martonosi. 2024. Distributed Quantum Computing via Integrating Quantum and Classical Computing. *Computer* 57, 4 (2024), 131–136. doi:10.1109/MC.2024.3360569
- Qisheng Wang, Riling Li, and Mingsheng Ying. 2022. Equivalence Checking of Sequential Quantum Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 9 (Sept. 2022), 3143–3156. doi:10.1109/tcad.2021.3117506
- Qisheng Wang, Junyi Liu, and Mingsheng Ying. 2021. Equivalence checking of quantum finite-state machines. *J. Comput. System Sci.* 116 (March 2021), 1–21. doi:10.1016/j.jcss.2020.08.004
- Shigeru Yamashita and Igor L. Markov. 2010. Fast equivalence-checking for quantum circuits. In *2010 IEEE/ACM International Symposium on Nanoscale Architectures*. 23–28. doi:10.1109/NANOARCH.2010.5510932
- Anocha Yimsiriwattana and Samuel J. Lomonaco Jr. 2004. Distributed quantum computing: a distributed Shor algorithm. In *Quantum Information and Computation II*, Eric Donkor, Andrew R. Pirich, and Howard E. Brandt (Eds.). SPIE. doi:10.1117/12.546504
- Mingsheng Ying. 2012. Floyd–Hoare Logic for Quantum Programs. *ACM Trans. Program. Lang. Syst.* 33, 6, Article 19 (Jan. 2012), 49 pages. doi:10.1145/2049706.2049708
- Mingsheng Ying and Yuan Feng. 2009. An Algebraic Language for Distributed Quantum Computing. *IEEE Trans. Comput.* 58, 6 (2009), 728–743. doi:10.1109/TC.2009.13
- Mingsheng Ying, Yuan Feng, Runyao Duan, and Zhengfeng Ji. 2009. An Algebra of Quantum Processes. *ACM Trans. Comput. Logic* 10, 3, Article 19 (apr 2009), 36 pages. doi:10.1145/1507244.1507249
- Mingsheng Ying, Li Zhou, and Yangjia Li. 2018. Reasoning about Parallel Quantum Programs. arXiv:1810.11334
- Mingsheng Ying, Li Zhou, Yangjia Li, and Yuan Feng. 2022. A proof system for disjoint parallel quantum programs. *Theoretical Computer Science* 897 (2022), 164–184. doi:10.1016/j.tcs.2021.10.025
- Zhicheng Zhang and Mingsheng Ying. 2024. Atomicity in Distributed Quantum Computing. arXiv:2404.18592 [quant-ph]
- Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational Verification of Quantum Programs. *Proc. ACM Program. Lang.* 7, POPL, Article 29 (jan 2023), 33 pages. doi:10.1145/3571222

A Expert Hand Crafted Rewrite Example

Here, we show an example of expert hand-crafted rewrites to distribute the ripple-carry adder circuit.

Example A.1 (Naive Hand-crafted Partition for the Ripple-Carry Adder). The program has two membranes, l and r . Initially, l holds the qubits $x[0]$, $y[0, 2]$, and $t[0, 2]$, while r holds the qubits $x[1]$, $y[2]$, and $t[2]$. The membranes share five 1-qubit quantum channels c , c_0 , c_1 , c_2 , and c_3 .

Processes:

$$NaL \triangleq x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{MAJ}.t[0] \sqcup y[1] \sqcup t[1] \leftarrow \text{MAJ}.c!t[1].c_0!t[0].c_1!y[1].c_2!x[0].c_3!y[0].0$$

$$NaR_3 \triangleq c_2?(x'[0]).c_3?(y'[0]).x'[0] \sqcup y'[0] \sqcup t'[0] \leftarrow \text{UMA}.0$$

$$NaR_2 \triangleq c_0?(t'[0]).c_1?(y'[1]).t'[0] \sqcup y'[1] \sqcup t'[1] \leftarrow \text{UMA}.NaR_3$$

$$NaR \triangleq c?(t'[1]).t'[1] \sqcup y[2] \sqcup t[2] \leftarrow \text{MAJ}.t[2] \sqcup x[1] \leftarrow \text{CX}.t'[1] \sqcup y[2] \sqcup t[2] \leftarrow \text{UMA}.NaR_2$$

Membranes:

$$\{\{NaL\}\}_l, \{\{NaR\}\}_r$$

Example A.2 (Expert Hand-crafted Partition for the Ripple-Carry Adder). The program has two membranes, l and r . Initially, l holds the qubits $x[0]$, $y[0, 2]$, and $t[0, 2]$, while r holds the qubits $x[1]$, $y[2]$, and $t[2]$. The membranes share two 1-qubit quantum channels c and c' .

Processes:

$$ExL_1 \triangleq c'?(t'[1]).t[0] \sqcup y[1] \sqcup t'[1] \leftarrow \text{UMA}.x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{UMA}.0$$

$$ExL \triangleq x[0] \sqcup y[0] \sqcup t[0] \leftarrow \text{MAJ}.t[0] \sqcup y[1] \sqcup t[1] \leftarrow \text{MAJ}.c!t[1].ExL_1$$

$$ExR \triangleq c?(t'[1]).t'[1] \sqcup y[2] \sqcup t[2] \leftarrow \text{MAJ}.t[2] \sqcup x[1] \leftarrow \text{CX}.t'[1] \sqcup y[2] \sqcup t[2] \leftarrow \text{UMA}.c'?t'[1].0$$

Membranes:

$$\{\{ExL\}\}_l, \{\{ExR\}\}_r$$

B The Formalism with Multiple Processes in a Membrane

Including the single-threaded parallelism will extend the AutoDisQ membrane with a multiset of processes. Here, we utilize the DisQ language [Chang et al. 2025] as a possibility of the extension. We demonstrate the possibility below.

B.1 Syntax for The DisQ Language

Unitary Expr	μ	
Bool Expr	B	
Channels	α	$::= a \mid c(n)$
Local Action	U	$::= \partial x(n) \mid \kappa \leftarrow \mu$
Communication Action	D	$::= \partial c(n) \mid \alpha!v \mid \alpha?(y)$
Process	R, T, M, N	$::= 0 \mid D.R \mid U.R \mid \text{if } (B) R \text{ else } T$
Membrane	P, Q	$::= \{\{\bar{R}\}\}_l \mid R\{\{\bar{T}\}\}_l$

Fig. 17. DisQ Syntax. We have the syntactic sugar: $\text{if } (B) \{R\}.T = \text{if } (B) R.T \text{ else } T$.

In DisQ, a distributed quantum program is described using a multiset of location-specific quantum processes, whose syntax (Figure 17) is captured by a membrane. There are two types of membrane descriptions: a multiset of processes $\{\{\bar{R}\}\}_l$ with location information (l) and an airlocked process associated with a membrane $(R\{\{\bar{T}\}\}_l)$. A process R , localized to a membrane, can be understood as a sequence of local quantum (U) or communication actions (D). We permit process algebraic message transmission operations (D typed, $\alpha!v$ and $\alpha?(y)$); they are the only communication actions that can perform direct message passing between different membranes, which can perform both classical and quantum message passing, depending on if the channels are classical or quantum. The operation

$$\begin{array}{lll}
\text{S-NEW} & & \text{S-OP} \\
(\varphi, \partial x(n).R) \xrightarrow{1} (\varphi \uplus \bigoplus_{j=0}^{n-1} \{x[j] : |0\rangle\}, R) & & (\varphi \uplus \{\kappa \boxplus \kappa' : \theta\}, \kappa \leftarrow \mu.R) \xrightarrow{1} (\varphi \uplus \{\kappa \boxplus \kappa' : \llbracket \mu \rrbracket^{|\kappa|} \theta\}, R) \\
\\
\text{S-SELF} & \text{S-IFT} & \text{S-IFF} \\
(\varphi, 0) \xrightarrow{1} (\varphi, 0) & (\varphi, \text{if } (1) R \text{ else } T) \xrightarrow{1} (\varphi, R) & (\varphi, \text{if } (0) R \text{ else } T) \xrightarrow{1} (\varphi, T) \\
\llbracket \mu \rrbracket^n (\sum_j z_j |d_j\rangle \beta_j) & \triangleq & \sum_j z_j (\llbracket \mu \rrbracket |d_j\rangle) \beta_j \quad \text{where } \forall j |d_j| = n \\
(\sum_i z_i |d_i\rangle \beta_i + \theta) \langle \kappa, b \rangle & \triangleq & \sum_i z_i |d_i\rangle \beta_i \quad \text{where } \forall i. |d_i| = |\kappa| \wedge \llbracket b[d_i/\kappa] \rrbracket = \text{true}
\end{array}$$

Fig. 18. DisQ single process semantic rules.

$\partial c(n)$ creates a quantum channel (c) . If we have the process $\partial c(n).R_1 \{\bar{T}_1\}_l, \partial c(n).R_2 \{\bar{T}_2\}_r$, interacting l and r , which collaboratively create an $2n$ -qubit Bell pair, each membrane shares an n qubit array, pointed to by the locus $\langle c[0, n] \rangle_l \boxplus \langle c[0, n] \rangle_r$, as in Figure 4. This is similar to π -calculus-style creation of new channels.

A local quantum operation can be a new blank $(|0\rangle)$ qubit array generator $\partial x(n)$, and a unitary operation $\kappa \leftarrow \mu$, applying a unitary operation μ to a local locus κ . In DisQ, we abstract away the detailed implementation of μ and assume that they can be analyzed by some systems describing quantum unitary circuits, e.g., VOQC [Hietala et al. 2023]. We also permit a classical conditional $\text{if } (B) R \text{ else } T$. The expression B is an arbitrary classical Boolean expression, implemented using bit-arithmetic, i.e., 1 as true and 0 as false.

B.2 DisQ Semantics

Quantum operations are probabilistic, and DisQ also enables single-location parallelism, modeled by probabilistic transitions. We model the DisQ semantics using a combination of Markov chains and Markov decision processes, which are divided into two categories: process-level and membrane-level semantics. The process level semantics is shown in Figure 18, which is expressed as a labeled transition system $(\varphi, R) \xrightarrow{p} (\varphi', T)$, where R and T are processes, φ and φ' are the pre- and post-local quantum states described in Figure 3, and p is the probability of the single step transition. The membrane level semantics defines the nondeterministic behaviors of a DisQ program, shown in Figure 19. It is formalized as a labeled transition system $(\Phi, \bar{P}) \xrightarrow{\xi, p} (\Phi', \bar{Q})$ where ξ (either l or $l.r$) captures the membrane locations (l or $l.r$) participating in the nondeterministic choice of the transition, p represents the probability of the transition, and Φ and Φ' are the global pre- and post-quantum states described in Figure 3.

Process Level Semantics. A DisQ process is a sequence of actions, and the rules in Figure 18 define the semantics for a local action prefixed in a process. Rule S-NEW creates new n blank qubits for locus $x[0, n]$. Rule S-SELF shows that the process semantics in a membrane is reflexive and can make a move to itself to preserve the stochastic property in a Markov chain, explained shortly below. Rule S-OP applies a quantum unitary operation to a locus κ 's quantum data. Here, the locus fragment κ to which the operation is applied must be prefixed in the locus $\kappa \boxplus \kappa'$ that refers to the entire quantum data θ . If not, we will first apply equivalence rewrites, explained in Section B.3 and [Li et al. 2024], to move κ to the front. With κ preceding the rest fragment κ' , the operation's semantic function $\llbracket \mu \rrbracket^n$ is then applied to κ 's position bases in the quantum value θ . More specifically, the function is only applied to the first n (equal to $|\kappa|$) basis bits of each basis-ket in the value while leaving the rest unchanged. For example, in Theorem 6.3, to apply H to the fragment $\langle x[1] \rangle_l$ of $\langle x[0, 3] \rangle_l \boxplus \langle y[0, 3] \rangle_u$, we use equivalence rewrites to ensure that $\langle x[1] \rangle_l$ is prefixed in the locus,

$$\begin{array}{c}
\text{S-COMP} \\
\frac{(\Phi, P) \xrightarrow{\xi, p} (\Phi', P')}{(\Phi, P, Q) \xrightarrow{\xi, p} (\Phi', P', Q)} \\
\\
\text{S-MEM} \\
\frac{n = |R, \bar{T}|}{(\Phi, \llbracket D.R, \bar{T} \rrbracket_l) \xrightarrow{l, \frac{1}{n}} (\Phi, D.R \llbracket \bar{T} \rrbracket_l)} \\
\\
\text{S-COMMC} \\
\frac{\Phi_1 = \{\langle c[0, n] \rangle_l \sqcup \langle c[0, n] \rangle_r : \bigotimes_{k=0}^{n-1} \sum_{d=0}^1 \frac{1}{\sqrt{2}} |d\rangle |d\rangle\}}{(\Phi \sqcup \Phi_1 \sqcup \{\langle \kappa \rangle_l \sqcup K : \theta\}, c(n) ! \kappa.R \llbracket \bar{M} \rrbracket_l, c(n) ? (x).T \llbracket \bar{N} \rrbracket_r) \xrightarrow{l, r, 1} (\Phi \sqcup \{\langle \kappa \rangle_r \sqcup K : \theta\}, \llbracket R, \bar{M} \rrbracket_l, \llbracket T[\kappa/x], \bar{N} \rrbracket_r)} \\
\\
\begin{array}{cc}
\text{S-REV} & \text{S-END} \\
(\Phi, R \llbracket \bar{T} \rrbracket_l) \xrightarrow{l, 1} (\Phi, \llbracket R, \bar{T} \rrbracket_l) & (\Phi, \llbracket \bar{0} \rrbracket_l) \xrightarrow{l, 1} (\Phi, \emptyset)
\end{array} \\
\\
\text{S-NEWCHAN} \\
(\Phi, \partial c(n).R \llbracket \bar{M} \rrbracket_l, \partial c(n).T \llbracket \bar{N} \rrbracket_r) \xrightarrow{l, r, 1} (\Phi \sqcup \bigoplus_{i=0}^{n-1} \{\langle c[i] \rangle_l \sqcup \langle c[i] \rangle_r : \sum_{d=0}^1 \frac{1}{\sqrt{2}} |d\rangle |d\rangle\}, \llbracket R, \bar{M} \rrbracket_l, \llbracket T[v/x], \bar{N} \rrbracket_r)
\end{array}$$

Fig. 19. Membrane-level semantic rules.

so it becomes $\langle x[1] \rangle_l \sqcup \langle x[0] \rangle_l \sqcup \langle x[2] \rangle_l \sqcup \langle y[0, 3] \rangle_u$. Rule S-IFT and S-IFF describe the semantics of classical conditionals.

Membrane Level Semantics. Figure 19 shows the membrane level semantics. A DisQ program is a set of membranes. We assume that the evaluation of the membrane set is *compositional* as S-COMP, i.e., every subset of the set can make a move. The transitions of processes in a membrane can be understood as a Markov chain, in the sense that each process has a chance to be selected to perform a location action or a communication action that requires an airlock step. This indicates that the chance of selecting any of the processes in a membrane equals $\frac{1}{n}$, where n is the number of processes in the membrane. We model the connection between the process and membrane-level semantics via the rules S-MEM and S-MOVE. The former handles the airlock mechanism for selecting a process in a membrane, ready for communication with another membrane, and the latter connects local action transitions with transition behaviors at the membrane level.

In S-MOVE, the locus $\langle \kappa \rangle_l \sqcup K$ is assumed to map to the data θ in the quantum state, and the prefixed action in R coincidentally is applied to the quantum datum pointed to by locus κ (in membrane l), which is guaranteed by the DisQ type system. As we mentioned in ??, the qubits mentioned in κ might be less than the qubits in the datum valuation θ , as the locus κ indicates that only the $|\kappa|$ length qubits prefixed in θ are able to be manipulate in the process R , while the remaining in θ is unreachable. The label $l, \frac{p}{n}$ indicates a nondeterministic choice of location l , where p is the probability of a one-step move in R . Rule S-REV permits the release of an airlock.

Note that, in DisQ, every membrane has a fixed number of processes in its lifetime. In rules S-MEM and S-MOVE, each probabilistic choice of performing a process has a probability $\frac{1}{n}$ where n is the number of processes in the membrane. To guarantee the equal distribution of the probabilistic choice of a process, we include rule S-SELF in Figure 18, as a $\emptyset$ process can make a move to itself. In the end, if every process in a membrane turns to $\emptyset$, rule S-END permits its termination.

Rule S-NEWCHAN creates a new quantum channel between the membranes l and r , which results in n pairs of Bell pair state connecting l and r , each of which is pointed to by the locus $\langle c[i] \rangle_l \sqcup \langle c[i] \rangle_r$ for $i \in [0, n)$. Rule S-COMMA performs a classical message communication inherited from traditional π -calculus [Milner et al. 1992], while S-COMMC performs a quantum message communication, assuming that the channel $c[0, n]$ is properly initialized as an n bitwidth Bell pair, as Φ_1 . Rules

S-NEWCHAN, S-COMMA, and S-COMMC transitions have labels $l.r.1$, meaning that the nondeterministic event happens across the l and r membranes. The probability 1 in the above three rules ensures that the transitions always happen.

DisQ intends to capture the probabilistic behavior of quantum computation via transition labels. Rules S-MEM and S-MOVE reveal that a membrane may contain multiple independent processes executing a series of events, each event execution occurring with the same probability.

B.3 DisQ Type System and Metatheories

We first discuss the constraints that the DisQ type system aims to guarantee. Quantum communication introduces additional restrictions due to the no-cloning principle, which imposes constraints on the system. For example, after the channel $c(1)$ and $x[i]$ are used in the send operation in the membrane l , one cannot apply additional operations to them in l anymore. We discover the following constraints.

Definition B.1 (Quantum Message and Channel Well-formed Constraints). We define the constraint property below for a correct DisQ program in a membrane.

- (1) Every quantum channel c must be initialized before being used, and used only once as a quantum message passing channel.
- (2) A quantum message θ cannot appear again in a later execution after it is sent.
- (3) A quantum operation $U(\bar{x})$ cannot have overlapping arguments in $\bar{x}$ and $U(\bar{x})$'s function body guarantees no-cloning.

The constraint (1) ensures that quantum channels are always one-time, while (2) ensures that a quantum message cannot be cloned but relocated. The constraint (3) ensures the no-cloning property in a sequential quantum operation. To ensure the constraints, DisQ utilizes a locus structure, a qubit collection indicating entangled groups, and ensures that qubits in two loci are not entangled. Before the order finding algorithm executes, we represent x and y registers as two separate loci $\langle x[0, n] \rangle_l$ and $\langle y[0, n] \rangle_r$, indicating that they are located in membrane l and are not entangled. After a single application on $x[0]$ in Theorem 3.1, we connect $x[0]$ with y registers to be a locus $\langle x[0] \rangle_r \sqcup \langle y[0, n] \rangle_r$, indicating that $x[0]$ is relocated in membrane r and joined the entangled group $\langle y[0, n] \rangle_r$, while the other locus becomes $\langle x[1, n] \rangle_l$. DisQ uses loci to characterize quantum resources in different membranes via typing, which is then used to partition the quantum state when evaluating programs, in order to guarantee Theorem B.1.

The DisQ type system also has two levels of typing judgments. The membrane level judgment is $\Omega; \Sigma \vdash \bar{P} \triangleright \Sigma'$, stating that $\bar{P}$ is well-typed under the kind and type environments Ω and Σ . The process level typing judgment is $\omega; \sigma \vdash \bar{R} \triangleright \sigma'$, stating that $\bar{R}$ is well-typed under the environments ω and σ . The membrane level kind environment Ω is a map $(l \rightarrow x \rightarrow g)$ and the process level kind environment ω is a map $(x \rightarrow g)$. The membrane level type environment Σ is a locus set, and the process level type environment σ is a set of local loci. The C-kind variables in a kind environment ω are populated through message receipt and quantum measurement operations, while the Q-kind variables are populated through a channel $\partial c(n)$ and qubit array $\partial x(n)$ creation operation. The type rules are in Figure 20. For every type rule, well-formed domains $(\Omega \vdash \text{dom}(\Sigma))$ (or $(\omega \vdash \text{dom}(\sigma))$) are required but hidden from the rules, such that every variable used in all loci of Σ (or σ) must appear in Ω . The type system enforces the three properties below.

Ensuring Proper Parameter Kinds and Scopes. The type system ensures scoping properties for variables and channels; e.g., quantum channels and variables have kind $Q(n)$, while classical channels and variables have kind C . Quantum channels and variables must be created before use and can be modified within a membrane. However, they cannot be referenced by operations from distinct membranes. Additionally, some operations, such as message sending and receiving, can

1520	T-OP	T-IF	T-BINDQ
1521	$\frac{\omega; \sigma \cup \{\kappa \sqcup \kappa'\} \vdash R \triangleright \sigma'}{\omega; \sigma \cup \{\kappa \sqcup \kappa'\} \vdash \kappa \leftarrow \mu.R \triangleright \sigma'}$	$\frac{\omega \vdash B : C \quad \omega; \sigma \vdash R \triangleright \sigma' \quad \omega; \sigma \vdash T \triangleright \sigma'}{\omega; \sigma \vdash \text{if } (B) \text{ R else } T \triangleright \sigma'}$	$\frac{\omega[c \mapsto Q(n)]; \sigma \cup \{c[0, n]\} \vdash R \triangleright \sigma'}{\omega; \sigma \vdash \partial c(n).R \triangleright \sigma'}$
1522			
1523	T-BINDQ1	T-MEA	T-SENDC
1524	$\frac{\omega[c \mapsto Q(n)]; \sigma \cup \{x[0, n]\} \vdash R \triangleright \sigma'}{\omega; \sigma \vdash \partial x(n).R \triangleright \sigma'}$	$\frac{\omega[x \mapsto C]; \sigma \cup \{\kappa'\} \vdash R \triangleright \sigma'}{\omega; \sigma \cup \{\kappa \sqcup \kappa'\} \vdash x \leftarrow \mathcal{M}(\kappa).R \triangleright \sigma'}$	$\frac{\omega \vdash a : C \quad \omega \vdash v : C \quad \omega; \sigma \vdash R \triangleright \sigma'}{\omega; \sigma \vdash a!v.R \triangleright \sigma'}$
1525			
1526	T-SENDQ	T-REVQ	T-REVC
1527	$\frac{\{c[0, n]\} \cup \{\kappa\} \subseteq \sigma \quad \kappa = n \quad \omega; \sigma \setminus (\{c[0, n]\} \cup \{\kappa\}) \vdash R \triangleright \sigma'}{\omega; \sigma \vdash c(n)! \kappa.R \triangleright \sigma'}$	$\frac{\{c[0, n]\} \subseteq \sigma \quad \omega[x \mapsto Q(n)]; \sigma \cup \{x[0, n]\} \setminus \{c[0, n]\} \vdash R \triangleright \sigma'}{\omega; \sigma \vdash c(n)?(x).R \triangleright \sigma'}$	$\frac{\omega \vdash a : C \quad \omega[x \mapsto C]; \sigma \vdash R \triangleright \sigma'}{\omega; \sigma \vdash a?(x).R \triangleright \sigma'}$
1528			
1529			
1530	T-MEM		
1531	$\frac{\text{has_mea}(\bar{R}) \quad \neg \text{has_mea}(\bar{T}) \quad \forall j \in [0, \bar{R}]. \Omega(l); \sigma_j \vdash \bar{R}_j \triangleright \sigma'_j \quad \Omega(l); \sigma \vdash \bar{T} \triangleright \sigma' \quad \Omega; \Sigma \vdash \bar{Q} \triangleright \Sigma'}{\Omega; \langle (\biguplus_{j \in [0, \bar{R}]} \sigma_j) \cup \Sigma \rangle_l \cup \Sigma \vdash \langle \llbracket \bar{R}, \bar{T} \rrbracket_l, \bar{Q} \rangle \triangleright \langle (\biguplus_{j \in [0, \bar{R}]} \sigma'_j) \cup \Sigma' \rangle_l \cup \Sigma'}$		
1532			
1533			
1534			
1535			
1536			
1537			
1538			
1539			
1540			
1541			
1542			
1543			
1544			
1545			
1546			
1547			
1548			
1549			
1550			
1551			
1552			
1553			
1554			
1555			
1556			
1557			
1558			
1559			
1560			
1561			
1562			
1563			
1564			
1565			
1566			
1567			
1568			

Fig. 20. DisQ type system. $\text{has_mea}(\bar{R})$: each $R \in \bar{R}$ syntactically contains a measurement op.

only refer to classical variables and channels. All these scoping properties are enforced by the type system. The Boolean ($\omega \vdash B : C$) and arithmetic ($\omega \vdash v : C$) expression checks (Section B.5) in rules T-IF, T-SENDC, and T-REVC, ensure that these expressions can only produce classical results and that their parameters are classical.

Ensuring Proper Locus Partitioning and Locality for the Well-formed Constraints. The constraints in Theorem B.1 are guaranteed by typing based on locus structure in DisQ. Our type system also ensures that loci are properly and disjointly partitioned in different membranes, and each membrane refers only to the permitted local loci. Rule T-MEM partitions loci by partitioning type environment, into pieces for different membranes, and ensures a properly separated analysis of different loci and quantum parameters in different membranes, where the structure $\langle \sigma \rangle_l$ is a subset of the type environment and represents a procedure of collecting all the loci referred to membrane P residing in location l , and type check P with the subset $\langle \sigma \rangle_l$.

In each membrane, a quantum channel and variable creation operations, in T-BINDQ and T-BINDQ1, create new channels and variables, as we push their local locus structures in the type environment, $\sigma \cup \{c[0, n]\}$ and $\sigma \cup \{x[0, n]\}$, and carry them in the further type checking. This guarantees the constraint (1) in Theorem B.1. The constraint (2) is guaranteed by rules T-SENDQ and T-REVQ, which remove quantum information being sent out, e.g., $\{c[0, n]\} \cup \{\kappa\}$ in T-SENDQ, in the type environment ($\sigma \setminus (\{c[0, n]\} \cup \{\kappa\})$) in the subsequent computation. Additionally, we also guarantee the bitwidth of the quantum channel ($c[0, n]$) is the same as the $|\kappa|$. In T-REVQ, when we receive quantum messages, we rename them as $\{x[0, n]\}$ and keep them in the type environment.

As in rules T-OP and T-MEA, we utilize locus structure, $\{\kappa \sqcup \kappa'\}$, to guarantee qubit disjointness in the quantum operation. In addition, we ensure that the measurement locus scopes are properly partitioned and preserved, e.g., rule T-IF requires the output type environments to be the same for the two branches (σ'). This indicates that if one branch contains a measurement on certain loci, the other branch must contain a similar measurement of these loci. In rule T-MEM, depending on whether or not a process contains measurement operations (has_mea), the quantum qubit resource sharing scheme is different. We collect all the local loci $(\biguplus_{j \in [0, |\bar{R}|]} \sigma_j) \cup \sigma$ in l , and partition it further into different disjoint union sets. For the processes $(\bar{R})$ with measurement operations, we type-check each R with a disjoint set σ_j . This forbids R from sharing qubits with other processes. If a process contains a measurement, it is not suitable for concurrent single-location behavior with

other processes, as this would allow different processes to refer to the same measured qubit. For the processes $\bar{T}$, having no measurements, we permit a shared qubit set σ .

Guiding Locus Equivalence and Rewriting. Sometimes, we want to shuffle the order of the locus for a quantum datum so that an operation can be correctly applied. The DisQ type system maintains the simultaneity of loci across type environments and quantum states through type-guided state rewrites formalized as equivalence relations. We saw examples of Shor’s algorithm in Theorem 3.1 that we might need to merge two entanglement groups or rearrange the qubit positions in loci. Such rewrites are formulated as type equivalence relations, which are associated with simultaneous quantum state rewrites. the details are introduced in [Li et al. 2024] and Section B.4. Here, we provide a taste of how such rewrites can happen. A locus represents a possibly entangled qubit group. In many cases, we need to utilize the locus information in the type environment to guide the equivalence rewrites of states guarded by the locus. We associate a state φ , with a type environment σ by sharing the same domain, i.e., $\text{dom}(\varphi) = \sigma$. Thus, the environment rewrites ($\preceq$) happening in σ gear the state rewrites ($\equiv$) in φ . One example rewrite is to add a qubit $x[j+1]$ to a local locus $x[0, j]$, and rewrite it to $\kappa(x[j-1] \sqcup x[0, j-1] \sqcup x[j])$, which can also cause the state rewrites happen accordingly as (from left to right):

$$\begin{aligned} \{x[0, j]\} &\cup \{x[j]\} &\preceq &\{x[0, j+1]\} &\preceq &\{\kappa\} \\ \{x[0, j] : \Sigma_{d=0}^1 \frac{1}{\sqrt{2}} |\bar{d}\rangle |1\rangle\} \cup \{x[j] : |0\rangle\} &\equiv &\{x[0, j+1] : \Sigma_{d=0}^1 \frac{1}{\sqrt{2}} |\bar{d}\rangle |1\rangle |0\rangle\} &\equiv &\{\kappa : \Sigma_{d=0}^1 \frac{1}{\sqrt{2}} |1\rangle |\bar{d}\rangle |0\rangle\} \end{aligned}$$

The correctness of our type system in Section B.3 is assumed to have well-formed domains below.

Definition B.2 (Well-formed locus domain). The domain of a environment Σ (or state Φ) is *well-formed*, written as $\Omega \vdash \Sigma$ (or $\text{dom}(\Phi)$), iff for every locus $K \in \Sigma$ (or $\text{dom}(\Phi)$):

- K is disjoint unioned, for every two ranges $\langle x[i, j] \rangle_l$ and $\langle y[i', j'] \rangle_l$ in K , $x[i, j] \cap y[i', j'] = \emptyset$.
- For every range $\langle x[i, j] \rangle_l \in K$, $\Omega(l)(x) = Q(n)$ and $[i, j] \subseteq [0, n]$.

Besides well-formed domain definition, we also require that states (Φ) being well-formed ($\Omega; \Sigma \vdash \Phi$), defined as follows. Here, we use $\Sigma(K)$ and $\Phi(K)$ to find the corresponding state entry pointed to by a locus K' , such that there exists $K_1 \cdot K' = K \sqcup K_1$.

Definition B.3 (Well-formed DisQ state). A state Φ is *well-formed*, written as $\Omega; \Sigma \vdash \Phi$, iff $\Sigma = \text{dom}(\Phi)$, $\Omega \vdash \Sigma$ (all variables in Φ are in Ω), and:

- For every $K \in \Sigma$, $\Phi(K) = \sum_{j=0}^m z_j |c_j\rangle \beta_j$, and for all j , $|K| = |c_j|$ and $\sum_{j=0}^m |z_j|^2 = 1$.

The DisQ Metatheories. We prove our type system’s preservation with respect to the semantics, assuming well-formedness. The theorems rely on the definitions of well-formed domains ($\Omega \vdash \Sigma$) and well-formed states ($\Omega; \Sigma \vdash \Phi$), shown in ???. With the type preservation theorem, we can show that DisQ programs respect the constraints in Theorem B.1.

Type preservation states that our type system ensures the three properties above and that the DisQ semantics can describe all different quantum operations without losing generality because we can always use the equivalence rewrites to rewrite the locus state in ideal forms.

THEOREM B.4 (TYPE PRESERVATION). If $\Omega \vdash \Sigma$, $\Omega; \Sigma \vdash \bar{P} \triangleright \Sigma'$, $(\Phi, \bar{P}) \xrightarrow{\alpha} (\Phi', \bar{P}')$, and $\Omega; \Sigma \vdash \Phi$, then there exists Ω_1 and Σ_1 , $\Omega_1; \Sigma_1 \vdash \bar{P}' \triangleright \Sigma'$ and $\text{dom}(\Omega_1) \subseteq \text{dom}(\Omega)$ and $\Sigma_1 \subseteq \Sigma$.

The term $\text{dom}(\Omega_1) \subseteq \text{dom}(\Omega)$ means that $\forall l. \text{dom}(\Omega_1)(l) \subseteq \text{dom}(\Omega)(l)$. The terms $\text{dom}(\Omega_1) \subseteq \text{dom}(\Omega)$ and $\Sigma_1 \subseteq \Sigma$ ensure that the post-environments Ω_1 and Σ_1 are consistent with the pre-environments Ω and Σ . With the type preservation theorem, we show that every DisQ evaluation maintains the constraints in Theorem B.1.

COROLLARY B.5 (CONSTRAINT SATISFACTION). Every well-typed program $\bar{P}$, as $\Omega \vdash \Sigma$ and $\Omega; \Sigma \vdash \bar{P} \triangleright \Sigma'$, satisfies the constraints in Theorem B.1, and every program evaluation $(\Phi, \bar{P}) \xrightarrow{\alpha} (\Phi', \bar{P}')$ maintains the constraint satisfaction in $\bar{P}'$.

B.4 DisQ Equivalence Relations

(a) Environment Equivalence

$$\begin{aligned} & \sigma \preceq \sigma \\ & \{\emptyset : \tau\} \uplus \sigma \preceq \sigma \\ & \{\kappa : \tau\} \uplus \sigma \preceq \{\kappa : \tau'\} \uplus \sigma \\ & \quad \text{where } \tau \sqsubseteq \tau' \\ & \{\kappa_1 \uplus s_1 \uplus s_2 \uplus \kappa_2 : \tau\} \uplus \sigma \preceq \{\kappa_1 \uplus s_2 \uplus s_1 \uplus \kappa_2 : \tau\} \uplus \sigma \\ & \{\kappa_1 : \tau\} \uplus \{\kappa_2 : \tau\} \uplus \sigma \preceq \{\kappa_1 \uplus \kappa_2 : \tau\} \uplus \sigma \\ & \{\kappa_1 \uplus \kappa_2 : \tau\} \uplus \sigma \preceq \{\kappa_1 : \tau\} \uplus \{\kappa_2 : \tau\} \uplus \sigma \end{aligned}$$

Permutation:

$$\begin{aligned} (q_1 \otimes q_2 \otimes q_3 \otimes q_4)^n (i \times k) &\triangleq q_1 \otimes q_3 \otimes q_2 \otimes q_4 \quad \text{where } |q_1| = n \wedge |q_2| = i \wedge |q_3| = k \\ (\Sigma_j z_j |c_j\rangle |c'_j\rangle \eta_j)^n (i \times k) &\triangleq \Sigma_j z_j |c_j\rangle |c'_j\rangle \eta_j \quad \text{where } |c_j| = n \wedge |c'_j| = i \wedge |c''_j| = k \end{aligned}$$

Join Product:

$$\begin{aligned} z_1 |c_1\rangle \bowtie z_2 |c_2\rangle &\triangleq (z_1 \cdot z_2) |c_1\rangle |c_2\rangle & \Sigma_{j=0}^n z_j |c_j\rangle \bowtie \Sigma_{k=0}^m z_k |c_k\rangle &\triangleq \Sigma_{j=0}^{n+m} z_j \cdot z_k |c_j\rangle |c_k\rangle \\ |c_1\rangle \bowtie \Sigma_j z_j \eta_j &\triangleq \Sigma_j z_j |c_1\rangle \eta_j & (|0\rangle + \alpha(r) |1\rangle) \bowtie \Sigma_j z_j \eta_j &\triangleq \Sigma_j z_j |0\rangle \eta_j + \Sigma_j (\alpha(r) \cdot z_j) |1\rangle \eta_j \end{aligned}$$

(b) State Equivalence

$$\begin{aligned} & \varphi \equiv \varphi \\ & \{\emptyset : q\} \uplus \varphi \equiv \varphi \\ & \{\kappa : q\} \uplus \varphi \equiv \{\kappa : q'\} \uplus \varphi \\ & \quad \text{where } q \equiv_{|\kappa|} q' \\ & \{\kappa_1 \uplus s_1 \uplus s_2 \uplus \kappa_2 : q\} \uplus \varphi \equiv \{\kappa_1 \uplus s_2 \uplus s_1 \uplus \kappa_2 : q'\} \uplus \varphi \\ & \quad \text{where } q' = q^{|\kappa_1|} (|s_1| \times |s_2|) \\ & \{\kappa_1 : q_1\} \uplus \{\kappa_2 : q_2\} \uplus \varphi \equiv \{\kappa_1 \uplus \kappa_2 : q'\} \uplus \varphi \\ & \quad \text{where } q' = q_1 \bowtie q_2 \\ & \{\kappa_1 \uplus \kappa_2 : \varphi\} \uplus \sigma \equiv \{\kappa_1 : \varphi_1\} \uplus \{\kappa_2 : \varphi_2\} \uplus \sigma \\ & \quad \text{where } \varphi_1 \bowtie \varphi_2 = \varphi \wedge |\varphi_1| = |\kappa_1| \end{aligned}$$

Fig. 21. DisQ type/state relations. $\cdot$ is math mult. Term $\Sigma^{n-m} P$ is a summation omitting the indexing details. $\otimes$ expands a Had array, as $\frac{1}{\sqrt{2^{n+m}}} \otimes_{j=0}^{n+m-2} q_j = (\frac{1}{\sqrt{2^n}} \otimes_{j=0}^{n-1} q_j) \otimes (\frac{1}{\sqrt{2^m}} \otimes_{j=0}^{m-1} q_j)$.

The DisQ type system maintains simultaneity via type-guided state rewrites, formalized as equivalence relations (see Figure 22). The equivalence relations happen in our type rules T-PAR and T-PARM in Figure 23. We only show the rewrite rules for local loci, and the loci with membrane structures can be manipulated through the merged rules in Figure 3, as well as a similar style of permutation rules in Section B.3. Other than the locus qubit position permutation being introduced, the types below associated with loci in the environment also play an essential role in the rewrites.

Quantum Type	$\tau ::=$	Nor		Had		EN
Quantum Value (Forms)	$q ::=$	w		$\frac{1}{\sqrt{2^n}} \otimes_{j=0}^{n-1} (0\rangle + \alpha(r_j) 1\rangle)$		$\Sigma_{j=0}^m w_j$

$\tau \sqsubseteq \tau$	$q \equiv_{ q } q$
Nor $\sqsubseteq$ EN	$ c\rangle \equiv_n \Sigma_{j=0}^0 c\rangle$
Had $\sqsubseteq$ EN	$\frac{1}{\sqrt{2^n}} \otimes_{j=0}^{n-1} (0\rangle + \alpha(r_j) 1\rangle) \equiv_n \Sigma_{j=0}^{2^{n-1}-1} \frac{\alpha(\Sigma_{k=0}^{n-1} r_k \cdot \mathbb{I}[j][k])}{\sqrt{2^n}} j\rangle$

(a) Subtyping

(b) Quantum Value Equivalence

$$x[n, n] \equiv \emptyset \quad \emptyset \uplus \kappa \equiv \kappa \quad |d_1\rangle |d_2\rangle \equiv |d_1 d_2\rangle \quad \langle q \uplus q' \rangle_l \equiv \langle q \rangle_l \uplus \langle q' \rangle_l \quad x[n, m] \equiv x[n, j] \uplus x[j, m] \text{ if } j \in [n, m]$$

(c) Locus Equivalence

Fig. 22. DisQ type/state relations.

$$\begin{array}{c}
\text{T-PAR} \\
\frac{\sigma \preceq \sigma' \quad \Omega; \sigma' \vdash e \triangleright \sigma''}{\Omega; \sigma_1 \cup \sigma \vdash e \triangleright \sigma_1 \cup \sigma''} \\
\\
\text{T-PARM} \\
\frac{\Sigma \preceq \Sigma' \quad \Omega; \Sigma' \vdash P \triangleright \Sigma''}{\Omega; \Sigma_1 \cup \Sigma \vdash P \triangleright \Sigma_1 \cup \Sigma''}
\end{array}$$

Fig. 23. Additional DisQ Type Rules.

The DisQ type system is inherited from the QAFNY type system [Li et al. 2024] with three different types. Quantum values are categorized into three different types: Nor, Had and EN. A *normal* value (Nor) is an array (tensor product) of single-qubit values $|0\rangle$ or $|1\rangle$. Sometimes, a (Nor)-typed value is associated with an amplitude z , representing an intermediate partial program state. A *Hadamard* (Had) typed value represents a collection of qubits in superposition but not entangled, i.e., an n -qubit array $\frac{1}{\sqrt{2}}(|0\rangle + \alpha(r_0)|1\rangle) \otimes \dots \otimes \frac{1}{\sqrt{2}}(|0\rangle + \alpha(r_{n-1})|1\rangle)$, can be encoded as $\frac{1}{\sqrt{2^n}} \otimes_{j=0}^{n-1} (|0\rangle + \alpha(r_j)|1\rangle)$, with $\alpha(r_j) = e^{2\pi i r_j}$ ($r_j \in \mathbb{R}$) being the *local phase*, a special amplitude whose norm is 1, i.e., $|\alpha(r_j)| = 1$. The most general form of n -qubit values is the *entanglement* (EN) typed value, consisting of a linear combination (represented as an array) of basis-kets, as $\sum_{j=0}^m z_j \beta_j \eta_j$, where m is the number of elements in the array. In DisQ, we *extend* traditional basis-ket structures in the Dirac notation to be the above form, so each basis-ket of the above value contains not only an amplitude z_j and a basis β_j but also a frozen basis stack η_j , storing bases not directly involved in the current computation. Here, β_j can always be represented as a single $|c_j\rangle$ by the equation in Figure 3. Every β_j in the array has the same cardinality, e.g., if $|c_0| = n$ ($\beta_0 = |c_0\rangle$), then $|c_i| = n$ ($\beta_i = |c_i\rangle$) for all j .

In DisQ, a locus represents a possibly entangled qubit group. From the study of many quantum algorithms [Ambainis 2004; Beauregard 2003; Childs et al. 2007; Häner et al. 2017; Magniez et al. 2005; Nielsen and Chuang 2011; Rigolin 2005; Shor 1994], we found that the establishment of an entanglement group can be viewed as a loop structure of incrementally adding a qubit to the group at a time, representing the entanglement's scope expansion. This behavior is similar to array splits and joins when we view quantum states as arrays. However, joining and splitting two EN-typed values are hard problems⁴. Another critical observation in studying many quantum algorithms is that entanglement group establishment usually involves splitting a qubit into a Nor/Had-typed value and joining it to an existing *tcht*-typed entanglement group. We manage these join-and-split pattern type-guided equations in DisQ, suitable for automated verification.

The semantics in Figure 18 assumes that the loci in quantum states can be in ideal forms, e.g., rule S-OP assumes that the target locus κ is always prefixed. This step is valid if we can rewrite (type environment partial order $\preceq$) the locus to the ideal form through rule T-PAR and T-PARM in Figure 20, which interconnectively rewrites the locus appearing in the state, through our state equivalence relation ($\equiv$), as the locus state simultaneity enforcement. The state equivalence rewrites have two components.

First, the type and quantum value forms have simultaneity in Figure 22, i.e., given a type τ_1 for a locus κ in a type environment (Σ), if it is a subtype ($\sqsubseteq$) of another type τ_2 , K 's value q_1 in a state (Φ) can be rewritten to q_2 that has the type τ_2 through state equivalence rewrites ($\equiv_n$) where n is the number of qubits in q_1 and q_2 . Both $\sqsubseteq$ and $\equiv_n$ are reflexive and types Nor and Had are subtypes of EN, which means that a Nor typed value ($|c\rangle$) and a Had typed value ($\frac{1}{\sqrt{2^n}} \otimes_{j=0}^{n-1} (|0\rangle + \alpha(r_j)|1\rangle)$) can be rewritten to an EN typed value. For example, a Had typed value $\frac{1}{\sqrt{2^n}} \otimes_{j=0}^{n-1} (|0\rangle + |1\rangle)$ can be

⁴The former is a Cartesian product; the latter is $\geq NP$ -hard.

rewritten to an EN type as $\sum_{i=0}^{2^n-1} \frac{1}{\sqrt{2^n}} |i\rangle$. If such a rewrite happens, we correspondingly transform $x[0, n]$'s type to EN in the type environment.

Second, type environment partial order ($\preceq$) and state equivalence ($\equiv$) also have simultaneity in Figure 21 for local loci, and the relations between loci can be derived based on the following rules, as well as permutations on $\sqcup$ operations.

$$\frac{\sigma \preceq \sigma'}{\langle \sigma \sqcup \sigma_1 \rangle_I \sqcup \Sigma \preceq \langle \sigma' \sqcup \sigma_1 \rangle_I \sqcup \Sigma} \quad \frac{\varphi \preceq \varphi'}{\langle \varphi \sqcup \varphi_1 \rangle_I \sqcup \Phi \preceq \langle \varphi' \sqcup \varphi_1 \rangle_I \sqcup \Phi}$$

Here, we associate a state Φ , with the type environment Σ by sharing the same domain, i.e., $\text{dom}(\Phi) = \text{dom}(\Sigma)$. Thus, the environment rewrites ($\preceq$) happening in Σ gear the state rewrites in Φ . In Figure 21, the rules of environment partial order and state equivalence are one-to-one corresponding. The first three lines describe the properties of reflective, identity, and subtyping equivalence. The fourth line enforces that the environment and state are close under locus permutation. After the equivalence rewrite, the position bases of ranges s_1 and s_2 are mutated by applying the function $q^{|\kappa_1|} \langle |s_1| \succ |s_2| \rangle$. One example is the following local locus rewrite from left to right, where we permute the two ranges $x[0, n]$ and $y[0, n]$.

$$\begin{aligned} \{x[0, n] \sqcup y[0, n] : \text{EN}\} &\preceq \{y[0, n] \sqcup x[0, n] : \text{EN}\} \\ \{x[0, n] \sqcup y[0, n] : \sum_{i=0}^{2^n-1} \frac{1}{\sqrt{2^n}} |i\rangle |a^i \% N\rangle\} &\equiv \{y[0, n] \sqcup x[0, n] : \sum_{i=0}^{2^n-1} \frac{1}{\sqrt{2^n}} |a^i \% N\rangle |i\rangle\} \end{aligned}$$

The last two lines in Figures 21a and 21b describe locus joins and splits, where the latter is an inverse of the former but much harder to perform practically. In the most general form, joining two EN-type states computes the Cartesian product of their basis-kets, shown at the bottom of Figure 21; such operations are computationally expensive in verification and validation. Fortunately, the join operations in most quantum algorithms are between a Nor/Had typed and an EN-typed state, Joining a Nor-typed and EN-typed state puts extra qubits in the right location in every basis-ket of the EN-typed state.

B.5 DisQ Kind Checking

$$\begin{array}{c} \frac{\omega(x) = C}{\omega \vdash x : \omega(x)} \quad \frac{\omega \vdash a_1 : C \quad \omega \vdash a_2 : C}{\omega \vdash_I a_1 + a_2 : C} \quad \frac{\omega \vdash a_1 : C \quad \omega \vdash a_2 : C}{\omega \vdash_I a_1 \cdot a_2 : C} \quad \frac{\omega \vdash a_1 : C \quad \omega \vdash a_2 : C}{\omega \vdash_I a_1 = a_2 : C} \\[10pt] \frac{\omega \vdash a_1 : C \quad \Omega \vdash a_2 : C}{\Omega \vdash_I a_1 < a_2 : C} \quad \frac{\omega \vdash b : C}{\omega \vdash \neg b : C} \end{array}$$

Fig. 24. Arith and Bool Kind Checking

The kind checking procedure $\omega \vdash - : C$ verifies if $-$ is a C kind term, based on the kind checking in [Li et al. 2024], and the rules for arithmetic and Boolean expressions are in Figure 24. The construct $-$ here refers to arithmetic, Boolean equations, or a statement.

C Quantum Teleportation and Classical Message Transmission Similarity

Quantum teleportation is a quantum network protocol that teleports information about a qubit to remote locations. This section shows a general use of quantum teleportation to teleport entanglement information. A key observation is that quantum entanglement is also a piece of information; thus, when teleporting a qubit, the possible entanglement associated with the qubit should also be kept by the remote qubits.

To demonstrate the case, we use the processes shown in Theorem C.1 to teleport a qubit $x[1]$ — currently entangled with $x[0]$ — from membrane l to r . The program first creates a shared quantum channel $c(1)$ between the two membranes, referred to by $\langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r$, and then teleport $x[1]$ to membrane r to store the information in $\langle x[1] \rangle_r$. The result should show that the entanglement between $\langle x[0] \rangle_l$ and $\langle x[1] \rangle_r$ is transferred to be an entanglement between $\langle x[0] \rangle_l$ and $\langle x[1] \rangle_r$.

Example C.1 (Quantum Teleportation Entanglement Preservation). The example has two membranes. The program code of membrane l is: $\{\partial c(1).c(1)!x[1].R\}_l$, and the program code of membrane r is: $\{\partial c(1).c(1)?(w).T\}_r$.

Membrane l has initially two qubit entangled state $x[0, 2] : z_0|00\rangle + z_1|11\rangle$.

$$K_e = \langle x[0, 2] \rangle_l \sqcup \langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r$$

The following provides the first few transition steps, where $\neg b$ is the bit-flip of the bit b . The R process in steps (3) and (4) refers to $R = i \leftarrow \mathcal{M}(b_1)x[1].i \leftarrow \mathcal{M}(b_2)c[0].a!b_1.a!2.0$.

- (1) $(\{\langle x[0, 2] \rangle_l : \sum_{b=0}^1 z_b |bb\rangle\}, \{\partial c(1).c(1)!x[1].R\}_l, \{\partial c(1).c(1)?(w).T\}_r)$
- (2) $\xrightarrow{l.1} (\{\langle x[0, 2] \rangle_l : \sum_{b=0}^1 z_b |bb\rangle\}, \partial c(1).c(1)!x[1].R\{\emptyset\}_l, \{\partial c(1).c(1)?(w).T\}_r)$
- (3) $\xrightarrow{r.1} (\{\langle x[0, 2] \rangle_l : \sum_{b=0}^1 z_b |bb\rangle\}, \partial c(1).c(1)!x[1].R\{\emptyset\}_l, \partial c(1).c(1)?(w).T\{\emptyset\}_r)$
- (4) $\xrightarrow{l.r.1} (\{\langle x[0, 2] \rangle_l : \sum_{b=0}^1 z_b |bb\rangle, \langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r : \frac{1}{\sqrt{2}} \sum_{b=0}^1 |bb\rangle\}, \{c(1)!x[1].R\}_l, \{c(1)?(w).T\}_r)$
- (5) $\xrightarrow{l.1} (\{\langle x[0, 2] \rangle_l : \sum_{b=0}^1 z_b |bb\rangle, \langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r : \frac{1}{\sqrt{2}} \sum_{b=0}^1 |bb\rangle\}, c(1)!x[1].R\{\emptyset\}_l, \{c(1)?(w).T\}_r)$
- (6) $\xrightarrow{r.1} (\{\langle x[0, 2] \rangle_l : \sum_{b=0}^1 z_b |bb\rangle, \langle c[0] \rangle_l \sqcup \langle c[0] \rangle_r : \frac{1}{\sqrt{2}} \sum_{b=0}^1 |bb\rangle\}, c(1)!x[1].R\{\emptyset\}_l, c(1)?(w).T\{\emptyset\}_r)$
- (7) $\xrightarrow{l.r.1} (\{\langle x[0] \rangle_l \sqcup \langle x[1] \rangle_r : \sum_{b=0}^1 z_b |bb\rangle\}, \{R\}_l, \{T[x[1]/w]\}_r)$

The above example shows that a Bell pair $\langle x[0, 2] \rangle_l$, which is a quantum channel itself, can be teleported to another membrane, indicated by the locus $\langle x[0] \rangle_l \sqcup \langle x[1] \rangle_r$. DisQ can demonstrate the behavior. Bell-pair teleportation, also known as quantum entanglement swapping, is useful for long-distance quantum message transmission. The approach is to first teleport half of a Bell pair to a remote location to establish a quantum channel over a longer distance.